ELABORATO FINALE DEL PROGETTO GYMMASTER INGEGNERIA DEL SOFTWARE A.A. 2025/2026.

Realizzato Da:

Carlo Curti e Samuele Cavallaro.

Sommario

1. Ideazione e analisi dei requisiti 3
1.1 Introduzione 3
1.2 Requisiti 3
1.3 Obiettivi e casi d'uso 4
1.4 Modello dei casi d'uso 5
UC1: Registra nuovo iscritto 5
UC2: Attiva abbonamento 6
UC3: Prenota lezione 7
UC4: Cancella prenotazione 8
UC5: Registra accesso 10
UC6: Visualizza abbonamenti in scadenza 11
UC7: Inserisci corso e lezioni 12
UC8: Visualizza calendario lezioni 13
UC9: Gestisci tipi di abbonamento 15
1.4 Documento di Visione 16
1.5 Regole di business 16
1.6 Glossario 16
2. Analisi Orientata agli Oggetti 17
2.1 Introduzione 17
2.2 Modello Di Dominio 18
2.3 SSD e Contratti 20
2.4 Diagramma ad Oggetti 30
3. Progettazione 31
3.1 Diagramma di Sequenza 32
3.2 Diagramma delle Classi 40
4. Testing 41

1. Ideazione e analisi dei requisiti

1.1 Introduzione

GymMaster rappresenta una soluzione moderna e flessibile pensata per supportare in modo efficace la gestione completa di una palestra. L'applicazione nasce con l'obiettivo di semplificare le attività quotidiane dello staff, migliorare l'esperienza degli iscritti e garantire un controllo dinamico sulle regole di dominio, come politiche di pricing, validità degli abbonamenti e modalità di accesso. Il sistema permette di amministrare in modo integrato iscrizioni, abbonamenti, prenotazioni delle lezioni di gruppo e registrazioni degli ingressi, offrendo strumenti intuitivi e adattabili alle esigenze operative della struttura. In questo documento vengono presentati in modo sintetico i servizi principali offerti da GymMaster e analizzate le necessità delle diverse parti coinvolte, con l'obiettivo di delineare una piattaforma completa, scalabile e orientata all'efficienza.

1.2 Requisiti

GymMaster è un applicativo finalizzato al miglioramento della gestione operativa di una palestra.

Dopo un'analisi preliminare dei requisiti, è stata individuata la necessità delle seguenti funzionalità:

- **Gestione iscritti:** Registrare e gestire l'anagrafica degli iscritti alla palestra.
- **Tipologie di abbonamento:** Definire e gestire diverse tipologie di abbonamento (durata, prezzo, condizioni di utilizzo).
- **Attivazione e rinnovo:** Attivare e rinnovare gli abbonamenti associati agli iscritti, con tracciamento delle date di validità.
- **Calendario lezioni di gruppo:** Gestire il calendario delle lezioni di gruppo, includendo corsi, istruttori, orari e capienza massima.
- **Prenotazioni lezioni:** Consentire la prenotazione delle lezioni di gruppo da parte degli iscritti tramite la segreteria.
- **Registrazione accessi:** Registrare gli accessi in palestra verificando in tempo reale la validità dell'abbonamento.
- **Scadenza abbonamenti:** Visualizzare gli abbonamenti in scadenza per facilitare le attività di rinnovo e comunicazione con gli iscritti.

1.3 Obiettivi e casi d’uso

Analizzando i requisiti riportati nel paragrafo precedente, sono stati individuati gli attori principali a cui è destinato il sistema e gli obiettivi che si intende portare a termine; da queste informazioni sono stati ricavati i casi d'uso principali.

Attore	Obiettivo	Caso d'uso
Addetto Segreteria	Gestire la registrazione di un nuovo iscritto nella palestra	UC1: Registra nuovo iscritto
Addetto Segreteria	Gestire l'attivazione di un abbonamento per un iscritto	UC2: Attiva abbonamento
Addetto Segreteria	Gestire la prenotazione di una lezione di gruppo per un iscritto	UC3: Prenota lezione
Addetto Segreteria	Gestire la cancellazione di una prenotazione	UC4: Cancella prenotazione
Addetto Segreteria	Gestire la registrazione dell'accesso di un iscritto in palestra	UC5: Registra accesso
Addetto Segreteria	Visualizzare gli abbonamenti in scadenza	UC6: Visualizza abbonamenti in scadenza
Gestore Palestra	Gestire l'inserimento di un nuovo corso con le relative lezioni	UC7: Inserisci corso e lezioni
Gestore Palestra	Visualizzare il calendario delle lezioni	UC8: Visualizza calendario lezioni
Gestore Palestra	Gestire le tipologie di abbonamento	UC9: Gestisci tipi abbonamento

1.4 Modello dei casi d'uso

Verranno illustrati di seguito tutti i vari casi d'uso che gradualmente, quelli descritti in formato breve, sono stati dettagliati.

UC1: Registra nuovo iscritto

Nome del caso d'uso	**UC1: Registra nuovo iscritto**
Portata	Applicazione GymMaster
Livello	Obiettivo utente
Attore primario	Addetto Segreteria
Parti interessate e interessi	- Addetto Segreteria: vuole registrare correttamente i dati del nuovo cliente; - Iscritto: vuole essere registrato per poter usufruire dei servizi della palestra.
Pre-condizioni	
Garanzia di successo	L'iscritto è registrato nel Sistema con tutti i dati anagrafici e un codice tessera univoco assegnato
Scenario principale di successo	1. Una persona si reca in segreteria per iscriversi alla palestra. 2. L'Addetto Segreteria sceglie l'attività "Registra nuovo iscritto". 3. Il Sistema richiede i dati anagrafici: nome, cognome, data di nascita, codice fiscale, telefono, email, indirizzo. 4. L'Addetto Segreteria inserisce i dati forniti dal cliente. 5. Il Sistema verifica che il codice fiscale non sia già presente (unicità). 6. Il Sistema genera un codice tessera univoco. 7. Il Sistema crea la scheda dell'iscritto e conferma l'operazione mostrando il riepilogo dei dati e il codice tessera. 8. L'Addetto comunica al cliente il codice tessera assegnato.
Estensioni	***a.** In un qualsiasi momento il Sistema e ha un arresto improvviso. 1. L'Addetto riavvia il software e ripristina lo stato precedente; 2. Il Sistema ripristina lo stato. **4a.** L'Addetto inserisce dati incompleti o non validi (es. email malformata). 1. Il Sistema segnala l'errore; 2. L'Addetto corregge i dati e ripete il passo 4. **5a.** Il codice fiscale è già presente nel sistema. 1. Il Sistema notifica "Iscritto già registrato" e mostra i dati esistenti; 2. L'Addetto può visualizzare la scheda esistente o annullare l'operazione.
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Quotidiana, legata all'afflusso di nuovi clienti
Varie	

UC2: Attiva abbonamento

Nome del caso d'uso	UC2: Attiva abbonamento
Portata	Applicazione GymMaster
Livello	Obiettivo utente
Attore primario	Addetto Segreteria
Parti interessate e interessi	- Addetto Segreteria: vuole attivare correttamente un abbonamento per l'iscritto e registrare il pagamento. - Iscritto: vuole ottenere un abbonamento valido per accedere alla palestra
Pre-condizioni	L'iscritto è già registrato nel sistema
Garanzia di successo	L'abbonamento è attivato, associato all'iscritto, con date di inizio e fine calcolate. Il sistema registra tutte le informazioni.
Scenario principale di successo	1. L'Iscritto si reca in segreteria per attivare un abbonamento. 2. L'Addetto Segreteria sceglie l'attività "Attiva abbonamento". 3. L'Addetto Segreteria inserisce nel Sistema il codice fiscale o il nome dell'iscritto. 4. Il Sistema mostra i dati dell'iscritto trovato. 5. L'Addetto Segreteria conferma l'iscritto selezionato. 6. Il Sistema mostra l'elenco delle tipologie di abbonamento disponibili con relativi prezzi e durate. 7. L'Iscritto comunica all'Addetto la tipologia di abbonamento desiderata. 8. L'Addetto Segreteria seleziona la tipologia di abbonamento. 9. L'Addetto Segreteria inserisce la data di inizio abbonamento (default: data odierna). 10. Il Sistema calcola automaticamente la data di fine in base alla durata del tipo abbonamento. 11. L'Addetto Segreteria conferma l'attivazione. 12. Il Sistema crea l'abbonamento, lo associa all'iscritto e conferma l'operazione. 13. L'Iscritto effettua il pagamento, lo stato dell'abbonamento viene impostato su ATTIVO, e l'iscritto se ne va.
Estensioni	***a.** In un qualsiasi momento il Sistema fallisce e ha un arresto improvviso. 1. L'Addetto riavvia il software e ripristina lo stato precedente del Sistema 2. Il Sistema ripristina lo stato. **3a.** L'Addetto inserisce un codice fiscale/nome non presente nel sistema. 1. Il Sistema notifica che l'iscritto non esiste 2. L'Addetto può registrare un nuovo iscritto (UC1) oppure reinserire i dati. **5a.** L'iscritto ha già un abbonamento attivo dello stesso tipo. 1. Il Sistema notifica che esiste già un abbonamento attivo

	2. L'Addetto può scegliere di rinnovare (creando un nuovo abbonamento che parte dalla scadenza del precedente) o annullare. **12a.** L'Addetto annulla l'operazione. 1. 1. Il Sistema annulla l'attivazione, nessun abbonamento viene creato.
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Quotidiana, legata all'afflusso di nuovi iscritti e rinnovi
Varie	

UC3: Prenota lezione

Nome del caso d'uso	**UC3: Prenota lezione**
Portata	Applicazione GymMaster
Livello	Obiettivo utente
Attore primario	Addetto Segreteria
Parti interessate e interessi	- Addetto Segreteria: vuole assicurare una corretta registrazione della prenotazione. - Iscritto: vuole prenotare un posto in una lezione di gruppo
Pre-condizioni	L'iscritto possiede un abbonamento attivo che include l'accesso alle lezioni di gruppo
Garanzia di successo	La prenotazione è registrata nel Sistema, il posto è riservato per l'iscritto, i posti disponibili della lezione sono aggiornati.
Scenario principale di successo	1. L'Iscritto si reca in segreteria per prenotare una lezione. 2. L'Iscritto comunica all'Addetto Segreteria il proprio nominativo e la lezione desiderata. 3. L'Addetto Segreteria sceglie l'attività "Prenota lezione" e inserisce il codice fiscale o nome dell'iscritto. 4. Il Sistema mostra i dati dell'iscritto e verifica che abbia un abbonamento valido con accesso alle lezioni. 5. Il Sistema mostra il calendario delle lezioni disponibili (prossimi giorni). 6. L'Addetto Segreteria seleziona il corso desiderato. 7. Il Sistema mostra le lezioni programmate per quel corso con data, ora, istruttore e posti disponibili. 8. L'Iscritto comunica all'Addetto la lezione specifica che vuole prenotare. 9. L'Addetto Segreteria seleziona la lezione. 10. Il Sistema verifica che ci siano posti disponibili. 11. Il Sistema crea la prenotazione, imposta il suo stato in EFFETTUATA, incrementa i posti occupati della lezione e conferma. 12. L'Addetto comunica all'Iscritto la conferma della prenotazione.

Estensioni	***a.** In un qualsiasi momento il Sistema fallisce e ha un arresto improvviso. 1. L'Addetto riavvia il software e ripristina lo stato precedente 2. Il Sistema ripristina lo stato. **4a.** L'iscritto non ha un abbonamento valido o l'abbonamento non include le lezioni. 1. Il Sistema notifica che l'iscritto non può prenotare lezioni 2. L'Addetto comunica all'iscritto la necessità di attivare/rinnovare l'abbonamento. **10a.** La lezione selezionata non ha posti disponibili (piena). 1. Il Sistema notifica "Lezione al completo" 2. L'Addetto comunica all'iscritto e torna al passo 7 per scegliere un'altra lezione. **12a.** L'iscritto ha già una prenotazione attiva per quella lezione. 1. Il Sistema notifica "Prenotazione già esistente" 2. L'Addetto comunica all'iscritto. **9a.** L'Addetto annulla l'operazione. 1. Il Sistema annulla, nessuna prenotazione viene creata.
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Alta, legata all'affluenza degli iscritti e al numero di lezioni offerte.
Varie	

UC4: Cancella prenotazione

Nome del caso d'uso	**UC4: Cancella prenotazione**
Portata	Applicazione GymMaster
Livello	Obiettivo utente
Attore primario	Addetto Segreteria
Parti interessate e interessi	- Addetto Segreteria: vuole gestire correttamente la cancellazione e liberare il posto nella lezione; - Iscritto: vuole annullare la propria prenotazione e liberarsi dall'impegno.
Pre-condizioni	- L'iscritto è registrato nel sistema; - L'iscritto ha almeno una prenotazione attiva.
Garanzia di successo	La prenotazione è annullata, il posto nella lezione è liberato e disponibile per altri iscritti
Scenario principale di successo	9. L'Iscritto si reca in segreteria per cancellare una prenotazione a una lezione. 10. L'Iscritto comunica all'Addetto Segreteria il proprio nominativo e la lezione da cui vuole cancellarsi. 11. L'Addetto Segreteria sceglie l'attività "Cancella prenotazione". 12. L'Addetto Segreteria inserisce nel Sistema il codice fiscale, il codice tessera o il nome dell'iscritto. 13. Il Sistema recupera i dati dell'iscritto e mostra l'elenco delle prenotazioni attive con i dettagli delle lezioni associate (corso, data, ora, sala).

	14. L'Addetto Segreteria seleziona la prenotazione da cancellare. 15. Il Sistema mostra i dettagli della prenotazione selezionata (corso, data, ora, sala, istruttore) e chiede conferma della cancellazione. 16. L'Addetto Segreteria conferma la cancellazione. 17. Il Sistema verifica che manchino almeno 2 ore all'inizio della lezione (regola R6). 18. Il Sistema imposta lo stato della prenotazione come ANNULLATA. 19. Il Sistema decrementa di 1 il contatore dei posti occupati della lezione. 20. Il Sistema conferma l'avvenuta cancellazione mostrando un messaggio di successo. 21. L'Addetto Segreteria comunica all'Iscritto che la prenotazione è stata cancellata.
Estensioni	***a.** In un qualsiasi momento il Sistema e ha un arresto improvviso. 3. L'Addetto riavvia il software e ripristina lo stato precedente; 4. Il Sistema ripristina lo stato. **4a.** L'Addetto inserisce un identificativo non presente nel sistema. 3. Il Sistema notifica "Iscritto non trovato"; 4. L'Addetto verifica i dati con l'Iscritto e ripete il passo 4. **5a.** L'iscritto non ha prenotazioni attive. 3. Il Sistema notifica "Nessuna prenotazione attiva trovata"; 4. L'Addetto comunica all'Iscritto che non ci sono prenotazioni da cancellare; 5. Il caso d'uso termina. **8a.** L'Addetto annulla l'operazione prima della conferma. 1. Il Sistema annulla l'operazione; 2. La prenotazione rimane attiva; 3. Il caso d'uso termina. **9a.** Mancano meno di 2 ore all'inizio della lezione (regola R5 non rispettata). 1. Il Sistema notifica "Impossibile cancellare: mancano meno di 2 ore all'inizio della lezione"; 2. L'Addetto comunica all'Iscritto che la cancellazione non è più possibile; 3. Il caso d'uso termina senza cancellazione. **6a.** L'Addetto seleziona una prenotazione già annullata (errore dati). 1. Il Sistema notifica "Prenotazione già annullata; 2. L'Addetto seleziona un'altra prenotazione o termina.
Requisiti speciali	Il controllo del tempo minimo (2 ore) deve essere calcolato rispetto alla data e ora effettiva della lezione, non del giorno corrente.
Elenco delle varianti tecnologiche e dei dati	• L'identificativo dell'iscritto può essere: codice tessera, codice fiscale o nome/cognome; • La data/ora della lezione può essere una data specifica o calcolata dal giorno della settimana ricorrente.
Frequenza di ripetizioni	Media, dipende dalle esigenze degli iscritti

Varie	• La regola R6 (cancellazione almeno 2 ore prima) è configurabile dal Gestore Palestra. • Le prenotazioni annullate rimangono nello storico per fini statistici.

UC5: Registra accesso

Nome del caso d'uso	**UC5: Registra accesso**
Portata	Applicazione GymMaster
Livello	Obiettivo utente
Attore primario	Addetto Segreteria
Parti interessate e interessi	- Addetto Segreteria: vuole registrare correttamente l'ingresso e verificare la validità dell'abbonamento. - Iscritto: vuole accedere alla palestra.
Pre-condizioni	L'iscritto è registrato nel sistema
Garanzia di successo	L'accesso è registrato nel Sistema con data e ora.
Scenario principale di successo	1. L'Iscritto si presenta all'ingresso della palestra. 2. L'Addetto Segreteria sceglie l'attività "Registra accesso". 3. L'Addetto Segreteria inserisce il codice fiscale, il nome o il codice tessera dell'iscritto. 4. Il Sistema recupera i dati dell'iscritto e il suo abbonamento attivo. 5. Il Sistema verifica che l'abbonamento sia valido (non scaduto, non sospeso). 6. Il Sistema registra l'accesso con data e ora correnti. 7. Il Sistema mostra conferma "Accesso consentito" con i dettagli dell'abbonamento. 8. L'Iscritto accede alla palestra.
Estensioni	***a.** In un qualsiasi momento il Sistema fallisce e ha un arresto improvviso. 1. L'Addetto riavvia il software e ripristina lo stato precedente 2. Il Sistema ripristina lo stato. **4a.** L'iscritto non è presente nel sistema. 1. Il Sistema notifica "Iscritto non trovato" 2. L'Addetto verifica i dati o propone la registrazione (UC1) **5a.** L'abbonamento è scaduto. 1. Il Sistema notifica "Abbonamento scaduto il [data]" 2. L'Addetto comunica all'iscritto e propone il rinnovo (UC2) 3. L'accesso viene negato. **5b.** L'abbonamento è sospeso. 1. Il Sistema notifica "Abbonamento sospeso" 2. L'accesso viene negato.
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Molto alta, ad ogni ingresso di un iscritto in palestra.

Varie	

UC6: Visualizza abbonamenti in scadenza

Nome del caso d'uso	**UC6: Visualizza abbonamenti in scadenza**
Portata	Applicazione GymMaster
Livello	Obiettivo utente
Attore primario	Addetto Segreteria
Parti interessate e interessi	- Addetto Segreteria: vuole identificare gli iscritti da contattare per proporre il rinnovo;
Pre-condizioni	Esistono abbonamenti nel sistema
Garanzia di successo	Viene visualizzato l'elenco degli abbonamenti in scadenza con i dati degli iscritti, permettendo azioni di follow-up
Scenario principale di successo	22. L'Addetto Segreteria vuole visualizzare gli abbonamenti in scadenza. 23. L'Addetto Segreteria sceglie l'attività "Visualizza abbonamenti in scadenza". 24. Il Sistema richiede il numero di giorni entro cui considerare la scadenza (default: 7 giorni). 25. L'Addetto Segreteria inserisce il numero di giorni desiderato (es. 7, 14, 30). 26. Il Sistema recupera tutti gli abbonamenti attivi con data di scadenza entro il periodo indicato.
Estensioni	***a.** In un qualsiasi momento il Sistema fallisce e ha un arresto improvviso. 5. L'Addetto riavvia il software e ripristina lo stato precedente; 6. Il Sistema ripristina lo stato. **5a.** Non esistono abbonamenti in scadenza nel periodo indicato. 1. Il Sistema notifica "Nessun abbonamento in scadenza nei prossimi [X] giorni"; 2. L'Addetto Segreteria può modificare il periodo di ricerca (torna al passo 4) o terminare. **4a.** L'Addetto Segreteria inserisce un valore non valido (es. numero negativo). 1. Il Sistema segnala "Valore non valido, inserire un numero positivo"; 2. L'Attore corregge e ripete il passo 4.
Requisiti speciali	• Deve essere possibile distinguere visivamente gli abbonamenti che scadono oggi o domani.
Elenco delle varianti tecnologiche e dei dati	• Periodi di scadenza comuni: 7 giorni, 14 giorni, 30 giorni.
Frequenza di ripetizioni	Quotidiana o settimanale, per attività di marketing e retention
Varie	• Si può configurare un invio automatico di email/SMS agli iscritti con abbonamento in scadenza (funzionalità futura). • Il report può includere statistiche sul tasso di rinnovo (funzionalità futura).

UC7: Inserisci corso e lezioni

Nome del caso d'uso	UC7: Inserisci corso e lezioni
Portata	Applicazione GymMaster
Livello	Obiettivo utente
Attore primario	Gestore Palestra
Parti interessate e interessi	- Gestore Palestra: vuole configurare l'offerta di corsi e programmare le lezioni nel calendario
Pre-condizioni	Esiste almeno un istruttore registrato nel sistema
Garanzia di successo	Il corso è creato con le relative lezioni programmate nel calendario, pronte per le prenotazioni
Scenario principale di successo	1. Il Gestore Palestra vuole inserire un nuovo corso con le relative lezioni. 2. Il Gestore sceglie l'attività "Inserisci corso". 3. Il Sistema richiede i dati del corso: nome, descrizione, istruttore predefinito. 4. Il Gestore inserisce i dati e seleziona l'istruttore dalla lista disponibile. 5. Il Sistema crea il corso e chiede se si vogliono programmare le lezioni. 6. Il Gestore conferma di voler inserire le lezioni. 7. Il Sistema richiede i dati della lezione: giorno della settimana, ora inizio, durata, sala, capienza massima, istruttore (default: quello del corso). 8. Il Gestore inserisce i dati della lezione. 9. Il Sistema verifica che la sala sia libera nella fascia oraria indicata. 10. Il Sistema crea la lezione e la associa al corso. 11. Il Sistema chiede se inserire altre lezioni. 12. I passi 7-11 vengono ripetuti finché il Gestore indica di aver finito. 13. Il Sistema conferma l'inserimento del corso con il riepilogo delle lezioni programmate.
Estensioni	***a.** In un qualsiasi momento il Sistema fallisce. 1. Il Gestore riavvia il software; 2. Il Sistema ripristina lo stato. **4a.** Non esistono istruttori nel Sistema. 1 Il Sistema notifica che occorre prima registrare un istruttore; 2 Il Gestore può creare un nuovo istruttore o annullare. **9a.** La sala è già occupata nella fascia orario indicata. 1. Il Sistema notifica "Sala non disponibile"; 2. Il Gestore modifica orario e ripete il passo 8.
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Occasionale, tipicamente a inizio stagione o quando si aggiungono nuovi corsi
Varie	
Requisiti speciali	• La visualizzazione deve essere chiara e ordinata per giorno/ora.

	• I posti disponibili devono essere evidenziati (es. colore verde se disponibili, rosso se esauriti). • Le lezioni passate devono essere distinguibili da quelle future.
Elenco delle varianti tecnologiche e dei dati	Modalità di visualizzazione supportate: • Giornaliera: tutte le lezioni di un giorno specifico; • Per corso: tutte le lezioni di un corso specifico; • Per istruttore: tutte le lezioni di un istruttore specifico;
Frequenza di ripetizioni	Molto alta, consultato frequentemente durante la giornata
Varie	

UC8: Visualizza calendario lezioni

Nome del caso d'uso	**UC8: Visualizza calendario lezioni**
Portata	Applicazione GymMaster
Livello	Obiettivo utente
Attore primario	Gestore Palestra
Parti interessate e interessi	- Gestore Palestra: vuole consultare il calendario per informare gli iscritti sulle lezioni disponibili e i posti liberi. Vuole avere una visione d'insieme della programmazione delle lezioni. - Iscritto (indirettamente): vuole conoscere le lezioni disponibili per prenotarsi.
Pre-condizioni	Esiste almeno una lezione programmata nel sistema.
Garanzia di successo	Il calendario delle lezioni viene visualizzato con tutte le informazioni richieste
Scenario principale di successo	1. Gestore Palestra vuole visualizzare il calendario delle lezioni; 2. Gestore Palestra sceglie l'attività "Visualizza calendario lezioni"; 3. Il Sistema mostra le opzioni di visualizzazione: per giorno, per corso, per istruttore; 4. Gestore Palestra seleziona la modalità di visualizzazione desiderata. 5. Il Sistema recupera le lezioni in base ai criteri selezionati. 6. Il Sistema mostra l'elenco delle lezioni con: nome corso, giorno, data, ora inizio, ora fine, durata, sala, istruttore, posti disponibili/totali, stato della lezione. 7. Gestore Palestra consulta le informazioni visualizzate. 8. Gestore Palestra può selezionare una lezione specifica per vedere i dettagli 9. Il Sistema mostra i dettagli completi della lezione selezionata. 10. Gestore Palestra può tornare alla vista calendario o terminare.
Estensioni	***a.** In un qualsiasi momento il Sistema e ha un arresto improvviso. 1. Gestore Palestra riavvia il software e ripristina lo stato precedente; 2. Il Sistema ripristina lo stato.

	5a. Gestore Palestra inserisce un identificativo non presente nel sistema. 1. Il Sistema notifica "Nessuna lezione trovata per i criteri selezionati"; 2. L'Attore può modificare i criteri di ricerca (torna al passo 4) o terminare. **4a.** L'iscritto non ha prenotazioni attive. 1. Il Sistema mostra l'elenco dei corsi attivi; 2. L'Attore seleziona un corso; 3. Il Sistema mostra solo le lezioni di quel corso (continua dal passo 6). **4b.** Gestore Palestra annulla l'operazione prima della conferma. 1. Il Sistema mostra l'elenco degli istruttori; 2. L'Attore seleziona un istruttore; 3. Il Sistema mostra solo le lezioni di quell'istruttore (continua dal passo 6). **4c.** Mancano meno di 2 ore all'inizio della lezione (regola R5 non rispettata). 1. Il Sistema richiede la data; 2. L'Attore inserisce la data desiderata; 3. Il Sistema mostra solo le lezioni di quel giorno (continua dal passo 6). **4d.** Gestore Palestra seleziona una prenotazione già annullata (errore dati). 1. Il Sistema mostra solo le lezioni in quella sala (continua dal passo 6). **8a.** L'Attore non seleziona alcuna lezione. 1. Il caso d'uso mostra dopo la consultazione del calendario. **9a.** La lezione selezionata non ha prenotazioni. 1. Il Sistema mostra i dettagli della lezione con la nota "Nessun iscritto prenotato; 2. Continua dal passo 10.
Requisiti speciali	• La visualizzazione deve essere chiara e ordinata per giorno/ora. • I posti disponibili devono essere evidenziati (es. colore verde se disponibili, rosso se esauriti). • Le lezioni passate devono essere distinguibili da quelle future.
Elenco delle varianti tecnologiche e dei dati	Modalità di visualizzazione supportate: • Giornaliera: tutte le lezioni di un giorno specifico; • Per corso: tutte le lezioni di un corso specifico; • Per istruttore: tutte le lezioni di un istruttore specifico; • Per sala: tutte le lezioni in una sala specifica.
Frequenza di ripetizioni	Molto alta, consultato frequentemente durante la giornata
Varie	• Il calendario può essere stampato o esportato (funzionalità futura). • Le lezioni annullate possono essere mostrate o nascoste in base a un'opzione di filtro.

	• Gestore Palestra può passare direttamente dalla visualizzazione calendario alla prenotazione di una lezione (collegamento con UC3).

UC9: Gestisci tipi di abbonamento

Nome del caso d'uso	UC9: Gestisci tipi di abbonamento
Portata	Applicazione GymMaster
Livello	Obiettivo utente
Attore primario	Gestore Palestra
Parti interessate e interessi	- Gestore Palestra: vuole definire l'offerta commerciale della palestra configurando le tipologie di abbonamento
Pre-condizioni	
Garanzia di successo	La tipologia di abbonamento è creata/modificata/disattivata nel Sistema e sarà disponibile per le attivazioni
Scenario principale di successo	14. Il Gestore Palestra vuole creare una nuova tipologia di abbonamento. 15. Il Gestore sceglie l'attività "Gestisci tipi abbonamento" e poi "Crea nuovo tipo". 16. Il Sistema richiede: nome, descrizione, durata (in giorni), prezzo, se include lezioni di gruppo (sì/no). 17. Il Gestore inserisce i dati richiesti. 18. Il Sistema verifica che non esista già un tipo abbonamento con lo stesso nome. 19. Il Sistema crea il tipo abbonamento e conferma l'operazione.
Estensioni	**5a.** Esiste già un tipo abbonamento con lo stesso nome. 3. Il Sistema segnala "Nome già esistente"; 4. Il Gestore modifica il nome e ripete. **4a.** Il Gestore inserisce dati non validi (es. durata negativa). 5. Il Sistema segnala l'errore; 6. Il Gestore corregge e ripete.
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Quotidiana, legata all'afflusso di nuovi clienti
Varie	
Scenari alternativi	Modifica tipo abbonamento: 1. Il Gestore sceglie "Modifica tipo abbonamento". 2. Il Sistema mostra l'elenco dei tipi esistenti. 3. Il Gestore seleziona il tipo da modificare. 4. Il Sistema mostra i dati attuali. 5. Il Gestore modifica i campi desiderati e conferma. 6. Il Sistema aggiorna il tipo abbonamento. Disattiva tipo abbonamento: 1. Il Gestore sceglie "Disattiva tipo abbonamento". 2. Il Sistema mostra l'elenco dei tipi attivi. 3. Il Gestore seleziona il tipo da disattivare.

	4. Il Sistema marca il tipo come "non attivo" (non sarà più selezionabile per nuove attivazioni, ma gli abbonamenti esistenti restano validi).

1.4 Documento di Visione

Durante la stesura di questo capitolo è stato redatto il documento di Visione che si trova nella cartella “Ideazione”.

1.5 Regole di business

Per il corretto utilizzo dell’applicazione devono essere rispettate queste regole di dominio.

ID	Regola	Modificabilità	Sorgente
R1	Un iscritto può prenotare una lezione solo se possiede un abbonamento attivo che include l'accesso alle lezioni di gruppo	Bassa	Politica interna della palestra
R2	Una prenotazione può essere effettuata solo se la lezione ha posti disponibili (postiOccupati < capienzaMax)	Bassa	Vincolo operativo
R3	L'accesso in palestra è consentito solo se l'iscritto possiede un abbonamento valido (non scaduto e non sospeso) alla data corrente	Bassa	Politica interna della palestra
R4	Un abbonamento si considera scaduto se la data corrente è successiva alla dataFine dell'abbonamento	Bassa	Regola di business
R5	La cancellazione di una prenotazione è consentita fino a 2 ore prima dell'inizio della lezione	Media	Politica interna della palestra
R6	Un iscritto non può avere più di una prenotazione attiva per la stessa lezione	Bassa	Vincolo di integrità

1.6 Glossario

Vengono qui riportati i termini più significativi e le relative definizioni.

Termine	Definizione
GymMaster	Rappresenta il sistema GymMaster
Iscritto	Persona registrata presso la palestra con i propri dati anagrafici, che può sottoscrivere abbonamenti
Abbonamento	Contratto che consente a un iscritto di accedere alla palestra e/o alle lezioni per un determinato periodo di tempo

TipoAbbonamento	Definizione astratta di un abbonamento (es. "Mensile", "Trimestrale", “Annuale”) con durata, prezzo e caratteristiche
Corso	Attività di gruppo ricorrente (es. Yoga, Spinning, CrossFit) tenuta da un istruttore
Lezione	Singola sessione programmata di un corso, con data, ora, sala, istruttore e capienza massima
Istruttore	Persona qualificata che conduce le lezioni di gruppo
Prenotazione	Atto con cui un iscritto riserva un posto in una lezione di gruppo
Accesso	Evento di ingresso di un iscritto nella palestra, registrato dal sistema
Segreteria	Punto di accoglienza della palestra dove l'addetto gestisce iscritti, abbonamenti e prenotazioni
Addetto Segreteria	Operatore che gestisce le operazioni quotidiane (iscrizioni, abbonamenti, prenotazioni, accessi)
Gestore Palestra	Amministratore responsabile della configurazione del sistema (corsi, istruttori, tipi abbonamento, calendario)
Sala	Ambiente della palestra destinato allo svolgimento delle lezioni di gruppo

2. Analisi Orientata agli Oggetti

2.1 Introduzione

Seguendo l’approccio iterativo ed evolutivo proposto da UP, lo sviluppo dell’applicazione è stato suddiviso in tre iterazioni. Questa scelta ha permesso di analizzare i requisiti in modo progressivo, riducendo il rischio di errori di progettazione o implementazione. Ogni iterazione ha affrontato un insieme specifico di problemi e casi d’uso.

Iterazione 1

In questa fase iniziale ci si è concentrati su:

- Approfondire il caso d’uso **UC1** e **UC9**.
- Progettare e implementare **UC1** e **UC9** e lo scenario di successo di **UC2**.

Iterazione 2

La seconda iterazione ha previsto:

- L’analisi approfondita dei casi d’uso **UC4, UC7** e **UC8**
- La progettazione e l’implementazione di questi casi d’uso, insieme allo scenario di successo di **UC3**

Iterazione 3

Nell'ultima iterazione si è proceduto con:

- L'analisi e l'implementazione del caso d'uso **UC6**.

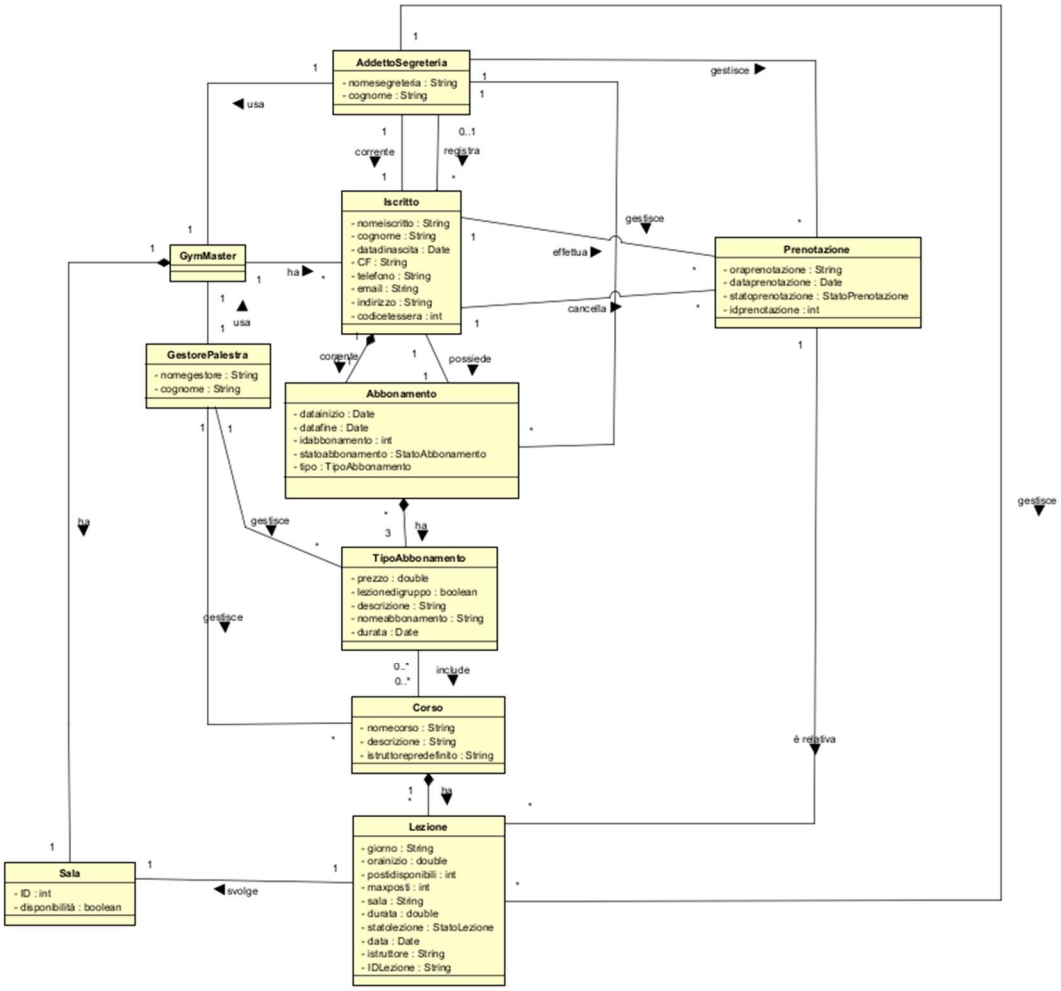
Metodo di lavoro

Ogni iterazione è iniziata con un'analisi dei requisiti orientata agli oggetti. Questo tipo di analisi mira a descrivere il dominio del problema attraverso concetti e relazioni tipici della programmazione a oggetti.

Per costruire questa descrizione sono stati utilizzati diversi strumenti, tra cui:

- **Modello di Dominio**, per rappresentare gli oggetti principali e le loro relazioni.
- **SSD (System Sequence Diagram)**, per descrivere le interazioni tra attori e sistema.
- **Contratti delle operazioni**, per definire in modo preciso il comportamento delle operazioni del sistema.

2.2 Modello Di Dominio



Sono state identificate le seguenti classi concettuali:

GymMaster: Rappresenta il sistema GymMaster

Gestore Palestra: Attore Primario. Amministratore responsabile della configurazione del sistema (corsi, istruttori, tipi abbonamento, calendario)

Addetto Segreteria: Attore Primario. Operatore che gestisce le operazioni quotidiane (iscrizioni, abbonamenti, prenotazioni, accessi)

Iscritto: Persona registrata presso la palestra con i propri dati anagrafici, che può sottoscrivere abbonamenti

Abbonamento: Contratto che consente a un iscritto di accedere alla palestra e/o alle lezioni per un determinato periodo di tempo

TipoAbbonamento: Definizione astratta di un abbonamento (es. "Mensile", "Trimestrale", "Annuale") con durata, prezzo e caratteristiche

Corso: Attività di gruppo ricorrente (es. Yoga, Spinning, CrossFit) tenuta da un istruttore

Lezione: Singola sessione programmata di un corso, con data, ora, sala, istruttore e capienza massima

Istruttore: Persona qualificata che conduce le lezioni di gruppo

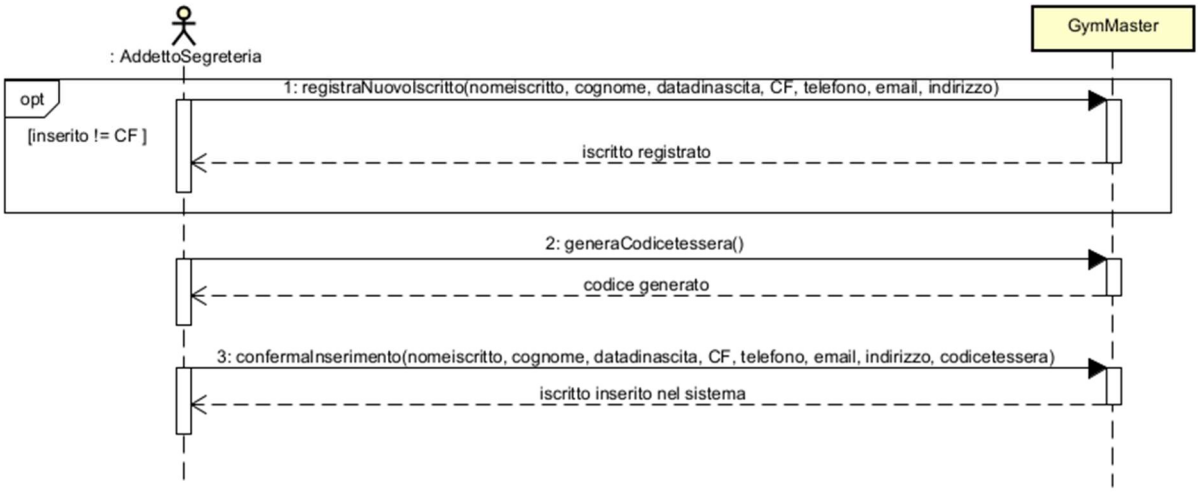
Sala: Ambiente della palestra destinato allo svolgimento delle lezioni di gruppo

2.3 SSD e Contratti

Proseguendo con l'analisi orientata agli oggetti, la fase successiva prevede la costruzione dei Diagrammi di Sequenza di Sistema (SSD). Questo strumento permette di rappresentare in modo preciso il flusso degli eventi di input e output associati ai vari casi d'uso affrontati in ciascuna iterazione. Inoltre, la maggior parte delle operazioni di sistema identificate all'interno degli SSD viene descritta in modo dettagliato attraverso i Contratti delle operazioni, che specificano precondizioni, postcondizioni e responsabilità del sistema. Grazie a questo processo, si ottiene una descrizione chiara e strutturata del comportamento del sistema, utile per guidare le fasi successive di progettazione e implementazione.

SSD E CONTRATTI ITERAZIONE 1:

2.3.1 UC1

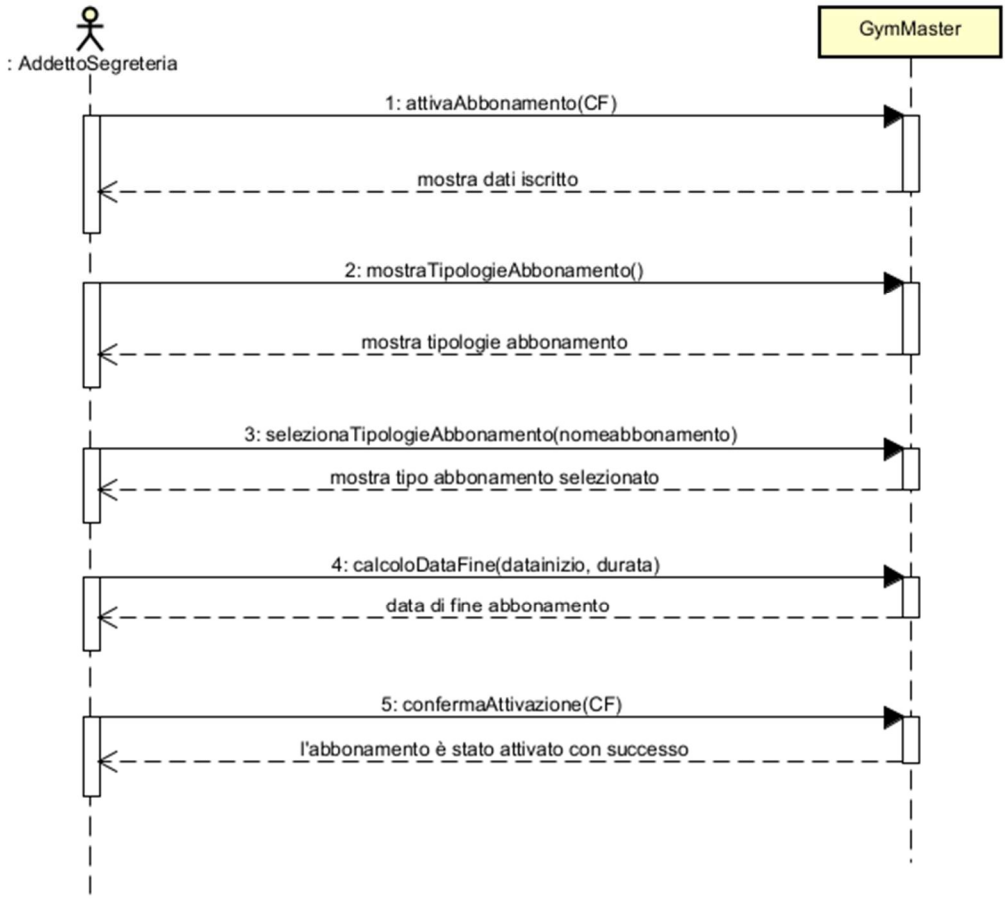


Campo	Descrizione
Operazione	registraNuovoIscritto(nome, cognome, datadinascita, CF, telefono, email, indirizzo)
Riferimenti	UC1 – Registra nuovo iscritto
Precondizioni	- Non esiste nel sistema un Iscritto con lo stesso CF
Postcondizioni	- È stata creata una nuova istanza i di Iscritto; - Gli attributi di i sono stati inizializzati; - i è stato associato a AdettoSegreteria tramite l'associazione "corrente"; - i è stato associato a Abbonamento tramite l'associazione "corrente"; - i è stato associato tramite l'associazione "ha" a GymMaster.

Campo	Descrizione
Operazione	generaCodiceTessera()
Riferimenti	UC1 – Registra nuovo iscritto
Precondizioni	- È stata creata una nuova istanza i di Iscritto
Postcondizioni	- Il Sistema genera il codice tessera; - Il codice tessera viene passato all'AddettoSegreteria

Campo	Descrizione
Operazione	confermaInserimento(nomeiscritto, cognome, datadinascita, CF, telefono, email, indirizzo, codicetessera)
Riferimenti	UC1 – Registra nuovo iscritto
Precondizioni	- È stato generato il codice tessera da associare all'iscritto
Postcondizioni	- L'istanza i di Iscritto è stata inserita nella Mappa Iscritti

2.3.2 UC2



Campo	Descrizione
Operazione	attivaAbbonamento(CF)
Riferimenti	UC2 – Attiva abbonamento
Precondizioni	- Esiste un Iscritto correntemente selezionato
Postcondizioni	- I dati dell'iscritto vengono visualizzati

Campo	Descrizione
Operazione	mostraTipologieAbbonamento()
Riferimenti	UC2 – Attiva abbonamento
Precondizioni	- I dati dell'iscritto sono stati visualizzati
Postcondizioni	- Vengono mostrate le tipologie di abbonamento

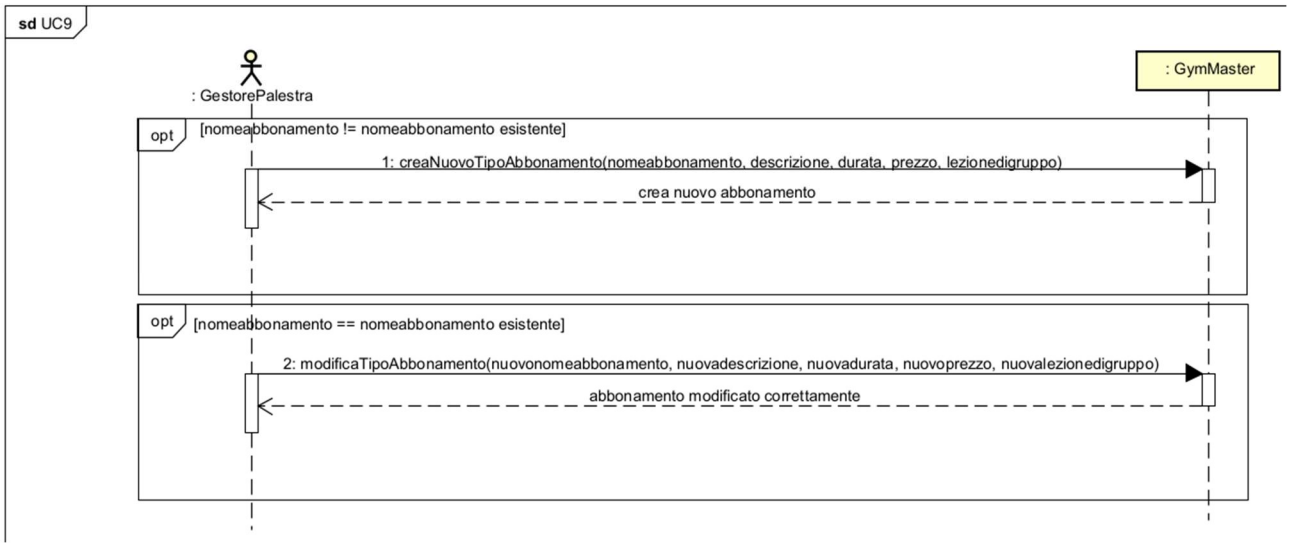
Campo	Descrizione
Operazione	selezionaTipologieAbbonamento(nomeabbonamento)
Riferimenti	UC2 – Attiva abbonamento
Precondizioni	- Esiste un Abbonamento correntemente selezionato
Postcondizioni	- Il Sistema mostra l'istanza ta selezionata dall'iscritto;

Campo	Descrizione
Operazione	calcoloDataFine(datainizio, durata)
Riferimenti	UC2 – Attiva abbonamento
Precondizioni	- È stata selezionata una tipologia di abbonamento;
Postcondizioni	- Viene calcolata datafine dal Sistema.

Campo	Descrizione
Operazione	confermaAttivazione(CF)
Riferimenti	UC2 – Attiva abbonamento

Precondizioni	- È stata calcolata la data di fine dell'abbonamento
Postcondizioni	- È stata creata una nuova istanza a di Abbonamento; - Gli attributi di a sono stati inizializzati; - a è stato associato a Iscritto tramite l'associazione "corrente"; - a è stato associato a AdettoSegreteria tramite l'associazione "gestisce"; - a è stato associato a TipoAbbonamento tramite l'associazione "ha";

2.3.3 UC9



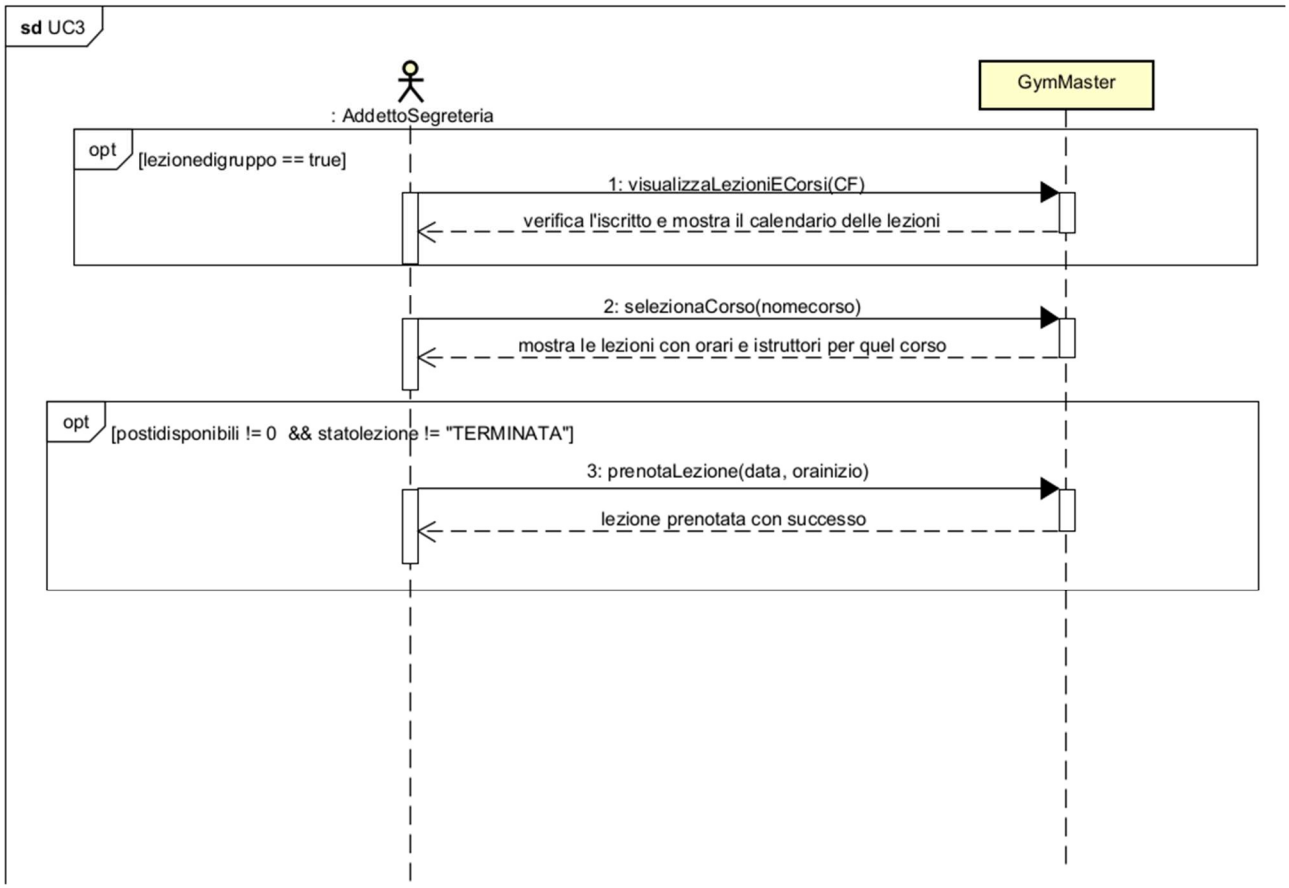
Campo	Descrizione
Operazione	creaNuovoTipoAbbonamento(nomeabbonamento, descrizione, durata, prezzo, lezionedigruppo)
Riferimenti	UC9 – Gestisci tipi di abbonamento
Precondizioni	- Il GestorePalestra ha effettuato l'accesso al Sistema
Postcondizioni	- È stata creata una nuova istanza ta di TipoAbbonamento; - Gli attributi di ta sono stati inizializzati; - ta è stato associato a Abbonamento tramite l'associazione "ha"; - ta è stato associato a GestorePalestra tramite l'associazione "gestisce".

Campo	Descrizione
Operazione	modificaTipoAbbonamento(nuovonomeabbonamento, nuovadescrizione, nuovadurata, nuovoprezzo, nuovalezionedigruppo)

Riferimenti	UC9 – Gestisci tipi di abbonamento
Precondizioni	- L'istanza ta è già esistente nel sistema
Postcondizioni	- Gli attributi dell'istanza ta vengono aggiornati - L'istanza ta è stata modificata correttamente

SSD E CONTRATTI ITERAZIONE 2:

2.3.4 UC3

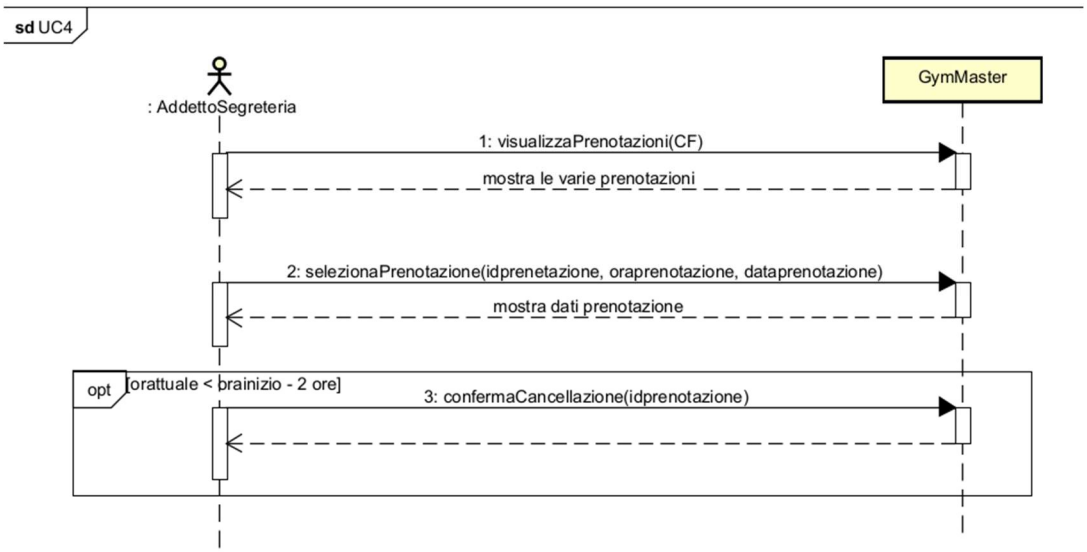


Campo	Descrizione
Operazione	visualizzaLezioniECorsi(CF)
Riferimenti	UC3 – Prenota lezione
Precondizioni	-
Postcondizioni	- Il Sistema verifica l'abbonamento dell'Iscritto in questione; - il Sistema mostra il calendario delle lezioni di ogni Corso.

Campo	Descrizione
Operazione	selezionaCorso(nomecorso)
Riferimenti	UC3 – Prenota lezione
Precondizioni	- L'abbonamento dell'Iscritto è stato verificato
Postcondizioni	- L'Iscritto seleziona il tipo di Corso che vuole frequentare; - Il Sistema mostra tutte le lezioni con i relativi orari e giorni riguardanti il Corso selezionato dall'Iscritto.

Campo	Descrizione
Operazione	prenotaLezione(data, orainizio)
Riferimenti	UC3 – Prenota lezione
Precondizioni	- L'Iscritto ha selezionato il Corso
Postcondizioni	- L'Iscritto seleziona la lezione che vuole frequentare; - Il Sistema procede con la prenotazione della lezione; - È stata creata una nuova istanza p di Prenotazione; - Gli attributi di p sono stati inizializzati; - p è stato associato a AddettoSegreteria tramite l'associazione "gestisce"; - p è stato associato a Iscritto tramite le associazioni "effettua" e "cancella"; - p è stato associato a Lezione tramite l'associazione "è relativa"; - Il Sistema aggiorna l'attributo postidiponibili dell'istanza l di Lezione.

2.3.5 UC4

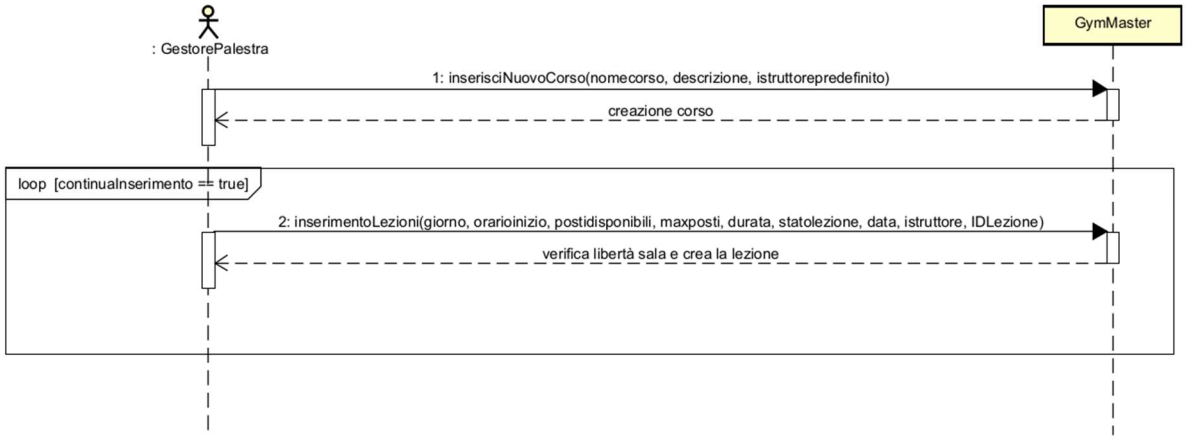


Campo	Descrizione
Operazione	visualizzaPrenotazioni(CF)
Riferimenti	UC4 – Cancella prenotazione
Precondizioni	- L'Iscritto ha effettuato almeno una prenotazione
Postcondizioni	- Il Sistema mostra le varie prenotazioni dell'Iscritto.

Campo	Descrizione
Operazione	selezionaPrenotazione(idprenetazione, oraprenotazione, dataprenotazione)
Riferimenti	UC4 – Cancella prenotazione
Precondizioni	-Il Sistema ha mostrato l'elenco delle prenotazioni effettuate dall'Iscritto
Postcondizioni	- L'Iscritto seleziona l'istanza p di Prenotazione che vuole cancellare; - Il Sistema mostra l'istanza p di Prenotazione che l'Iscritto vuole cancellare.

Campo	Descrizione
Operazione	confermaCancellazione(idprenotazione)
Riferimenti	UC4 – Cancella prenotazione
Precondizioni	-L'Iscritto ha selezionato la prenotazione da cancellare
Postcondizioni	- Il Sistema chiede conferma all'Iscritto sulla prenotazione da cancellare; - L'Iscritto conferma la cancellazione dell'istanza p di Prenotazione; - Il Sistema cancella l'istanza p di Prenotazione; - Il Sistema aggiorna l'attributo postidisponibili dell'istanza l di Lezione.

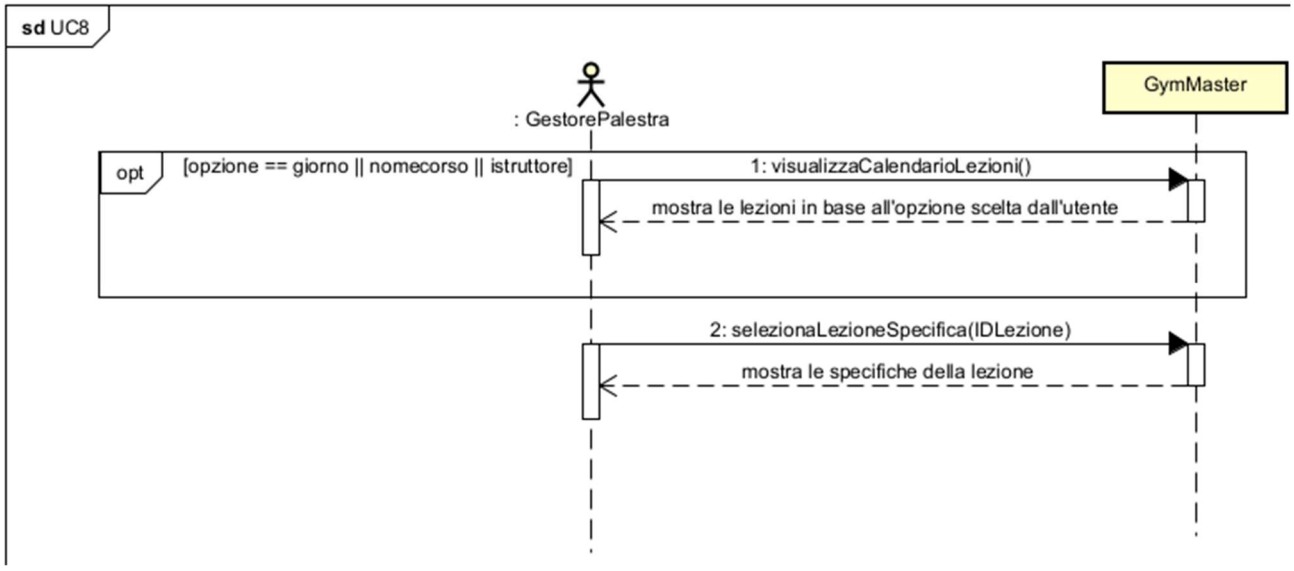
2.3.6 UC7



Campo	Descrizione
Operazione	inserisciNuovoCorso(nomecorso, descrizione, istruttorepredefinito)
Riferimenti	UC7 – Inserisci corso e lezioni
Precondizioni	-
Postcondizioni	- È stata creata una nuova istanza c di Corso; - Gli attributi di c sono stati inizializzati; - c è stato associato a TipoAbbonamento tramite la relazione “include”; - c è stato associato a GestorePalestra tramite la relazione “gestisce”; - c è stato associato a Lezione tramite la relazione “ha”.

Campo	Descrizione
Operazione	inserimentoLezioni(giorno, orarioinizio, postidisponibili, maxposti, sala, durata, statolezione, data, istruttore, IDLezione)
Riferimenti	UC7 – Inserisci corso e lezioni
Precondizioni	- È stata creata un’istanza c di Corso
Postcondizioni	- Il Sistema mostra se la Sala è libera; - È stata creata una nuova istanza l di Lezione; - Gli attributi di l sono stati inizializzati; - l è stato associato a Corso tramite la relazione “ha”; - l è stato associato a Sala tramite la relazione “svolge”; - l è stato associato a AddettoSegreteria tramite la relazione “gestisce”; - l è stato associato a Prenotazione tramite la relazione “è relativa”;

2.3.7 UC8

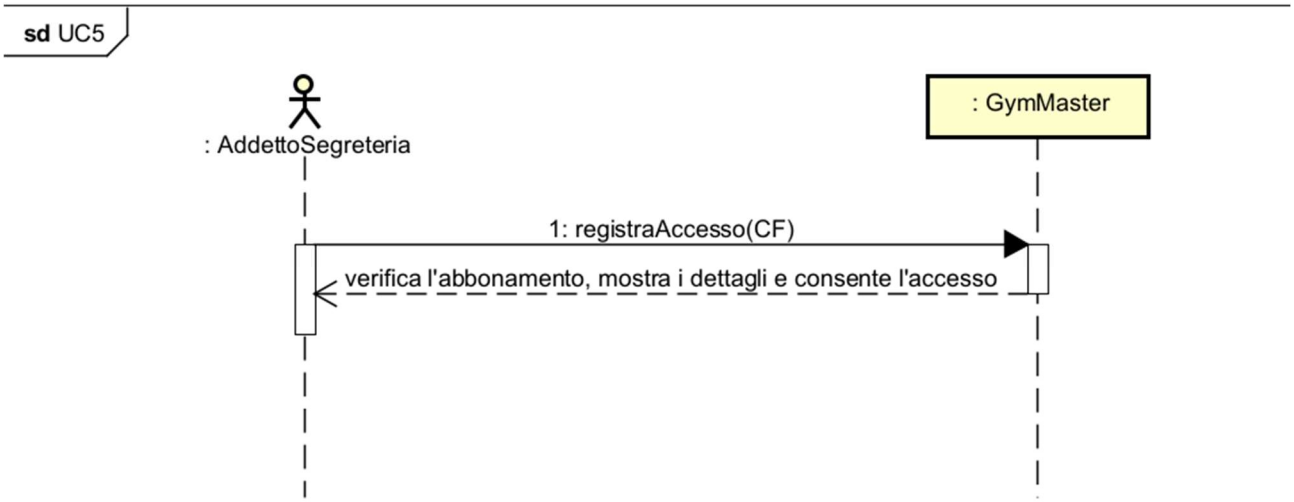


Campo	Descrizione
Operazione	visualizzaCalendarioLezioni()
Riferimenti	UC8 – Visualizza calendario lezioni
Precondizioni	-Sono state create delle lezioni
Postcondizioni	- L'AddettoSegreteria vuole mostrato il calendario delle lezioni; - Il Sistema gli mostra le opzioni di visualizzazione; - L'AddettoSegreteria sceglie l'opzione di visualizzazione; - Il Sistema mostra il calendario delle lezioni.

Campo	Descrizione
Operazione	selezionaLezione (IDLezione)
Riferimenti	UC8 – Visualizza calendario lezioni
Precondizioni	-Il Sistema ha mostrato il calendario delle lezioni all'AddettoSegreteria
Postcondizioni	- L'AddettoSegreteria seleziona l'istanza l di Lezione; - Il Sistema mostra le specifiche dell'istanza l di Lezione.

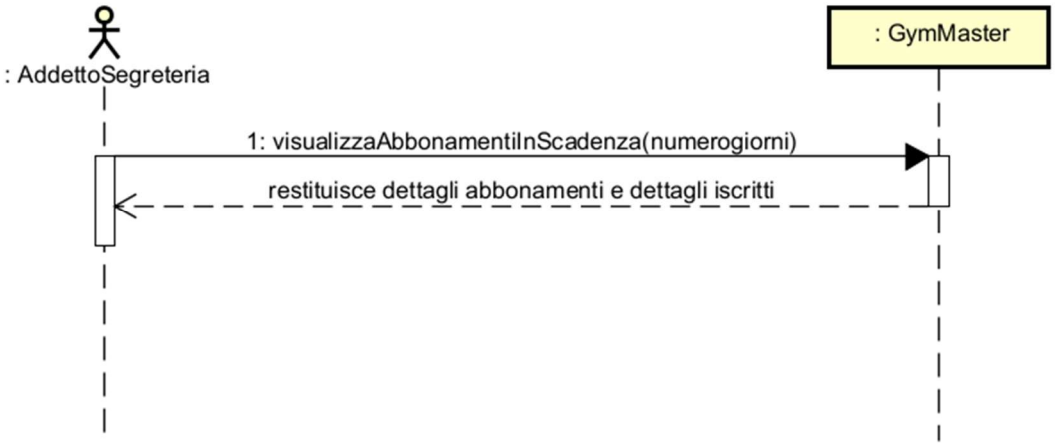
SSD E CONTRATTI ITERAZIONE 3:

2.3.8 UC5



Campo	Descrizione
Operazione	registraAccesso(codicetessera)
Riferimenti	UC5 – Registra Accesso
Precondizioni	-
Postcondizioni	- Il Sistema recupera i dati dell'Iscritto che sta accedendo in palestra e del suo Abbonamento; - Il Sistema verifica l'attributo statoabbonamento di Abbonamento; - Il Sistema registra l'accesso; - L'Iscritto accede in palestra.

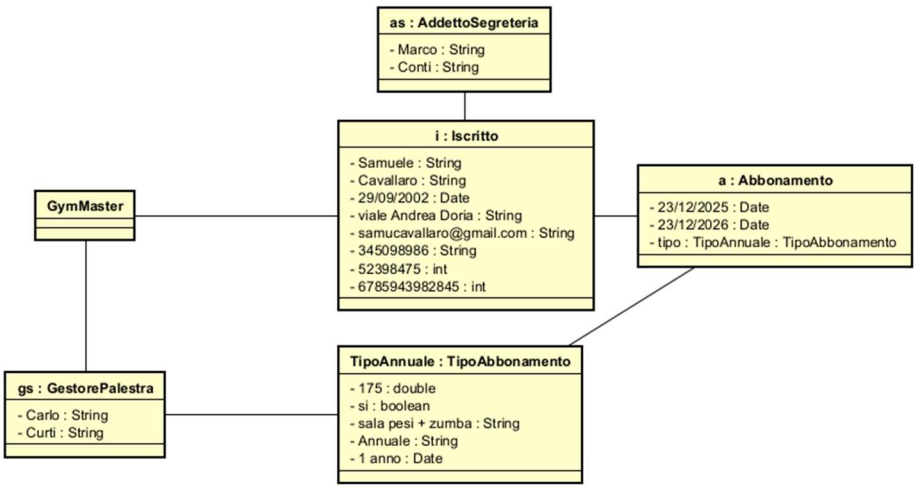
2.3.8 UC6



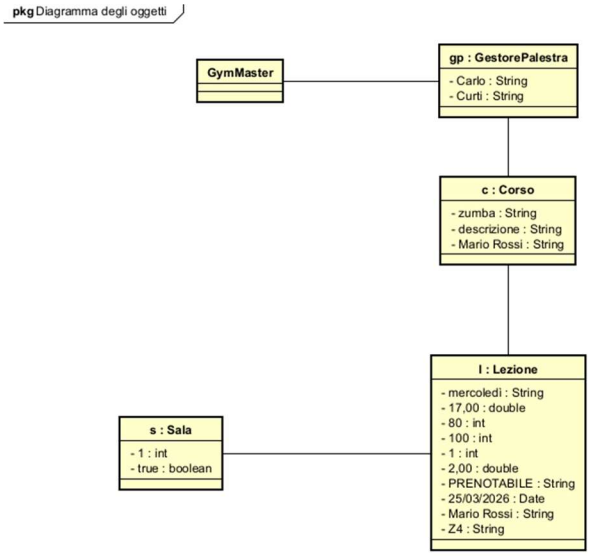
Campo	Descrizione
Operazione	visualizzaAbbonamentiInScadenza(numerogiorni)
Riferimenti	UC6 – Visualizza abbonamenti in scadenza
Precondizioni	- Esistono abbonamenti nel Sistema
Postcondizioni	- L'Addetto Segreteria inserisce il numerogiorni entro cui considerare la scadenza; - Vengono visualizzati tutti gli attributi delle istanze a degli Abbonamenti in scadenza con i relativi attributi dell'istanza i di Iscritto;

2.4 Diagramma ad Oggetti

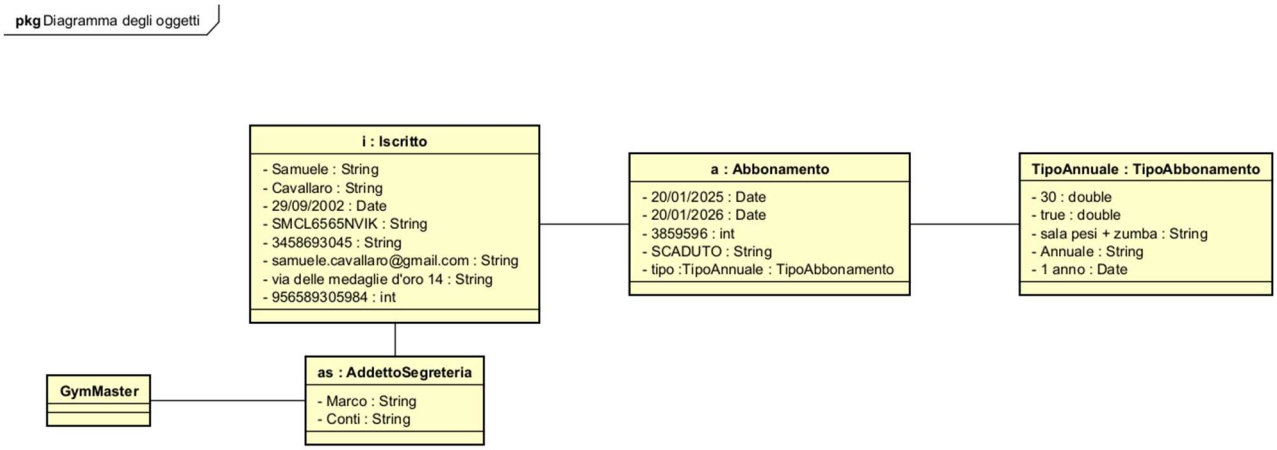
ITERAZIONE 1: Relativamente al caso d'uso 2 (Attiva Abbonamento). È stato redatto il seguente diagramma ad oggetti per una migliore comprensione del dominio.



ITERAZIONE 2: Relativamente al caso d'uso 7 (Inserisci Corso e Lezioni). È stato redatto il seguente diagramma ad oggetti per una migliore comprensione del dominio.



ITERAZIONE 3: Relativamente al caso d'uso 5 (Registra Accesso). È stato redatto il seguente diagramma ad oggetti per una migliore comprensione del dominio.



3. Progettazione

Durante la fase di progettazione, per ciascuna iterazione sono stati sviluppati i **diagrammi di sequenza (SD)** e il **diagramma delle classi di progetto (DCD)**. Questi, sono stati progressivamente ampliati man mano che venivano implementati nuovi casi d'uso e introdotti i pattern necessari. In questo modo la progettazione si è evoluta insieme al sistema, integrando gradualmente sia le funzionalità richieste sia le soluzioni architetturali adottate.

I pattern GoF che sono stati utilizzati sono:

- **Factory Method;**
- **Observer.**

Il **Factory Method** è stato introdotto a partire dall'interazione 1, in quanto prima la creazione degli abbonamenti avveniva direttamente all'interno di GymMaster nel metodo confermaAttivazione(). Il codice conteneva uno switch per determinare la durata dell'abbonamento in base alla tipologia selezionata, calcolando manualmente la data di fine, e ogni volta che si voleva aggiungere una nuova tipologia di abbonamento (ad esempio "Settimanale" o "Semestrale"), era necessario modificare il codice esistente di GymMaster, aggiungendo un nuovo ramo condizionale (mentre adesso per aggiungere una nuova tipologia, basta creare una nuova classe che estende AbbonamentoFactory, senza toccare il codice esistente). Questo aumentava il rischio di introdurre errori nel codice già funzionante, e inoltre le regole specifiche di ogni tipologia (durata 30, 90 o 365 giorni, calcolo della data di fine, impostazione dello stato iniziale) erano tutte concentrate in un unico metodo di GymMaster, rendendo il codice difficile da leggere e manutenere. Infine ogni factory può essere testata indipendentemente (MensileFactoryTest, TrimestraleFactoryTest, AnnualeFactoryTest), verificando che produca abbonamenti con la durata corretta.

Con il Factory Method, ogni ConcreteFactory (MensileFactory, TrimestraleFactory, AnnualeFactory) incapsula la propria logica di creazione:

- MensileFactory sa che deve aggiungere 30 giorni alla data di inizio
- TrimestraleFactory sa che deve aggiungere 90 giorni
- AnnualeFactory sa che deve aggiungere 365 giorni

Il metodo confermaAttivazione() di GymMaster ora non conosce più i dettagli di creazione, si limita a chiamare abbonamentoFactory.creaAbbonamento(iscritto, dataInizio) e riceve un abbonamento già configurato. La selezione della factory avviene una sola volta in selezionaTipologieAbbonamento() tramite uno switch, ma la logica di creazione vera e propria è completamente delegata alla factory.

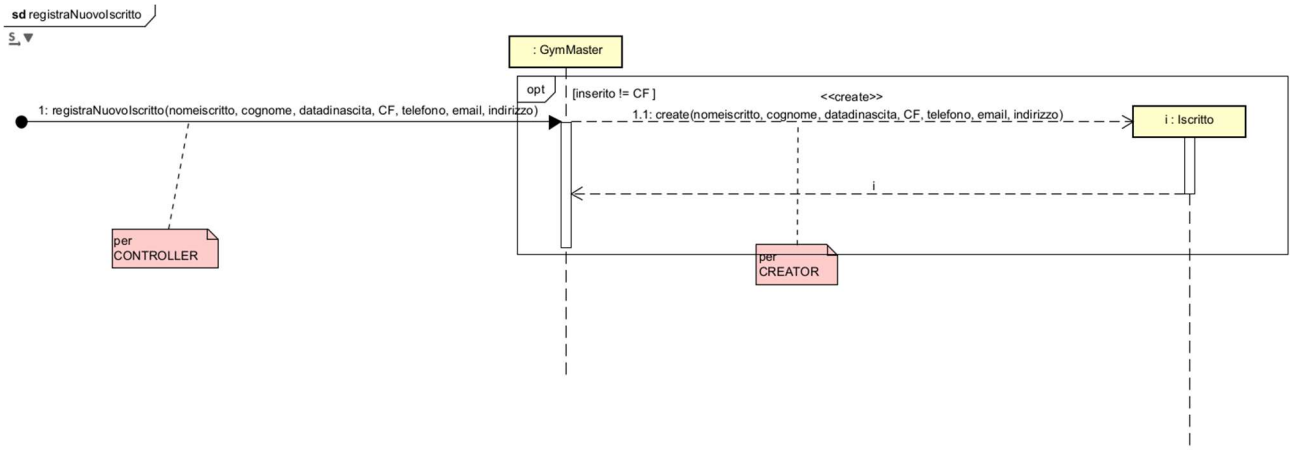
Mentre l'**Observer** è stato introdotto a partire dall'interazione 2, con la classe Lezione che estende Subject e mantiene una lista di observer. Quando i posti disponibili cambiano (sia per una prenotazione che per una cancellazione), Lezione chiama automaticamente notifyObserver(), che a sua volta invoca il metodo update() su tutti gli observer registrati.

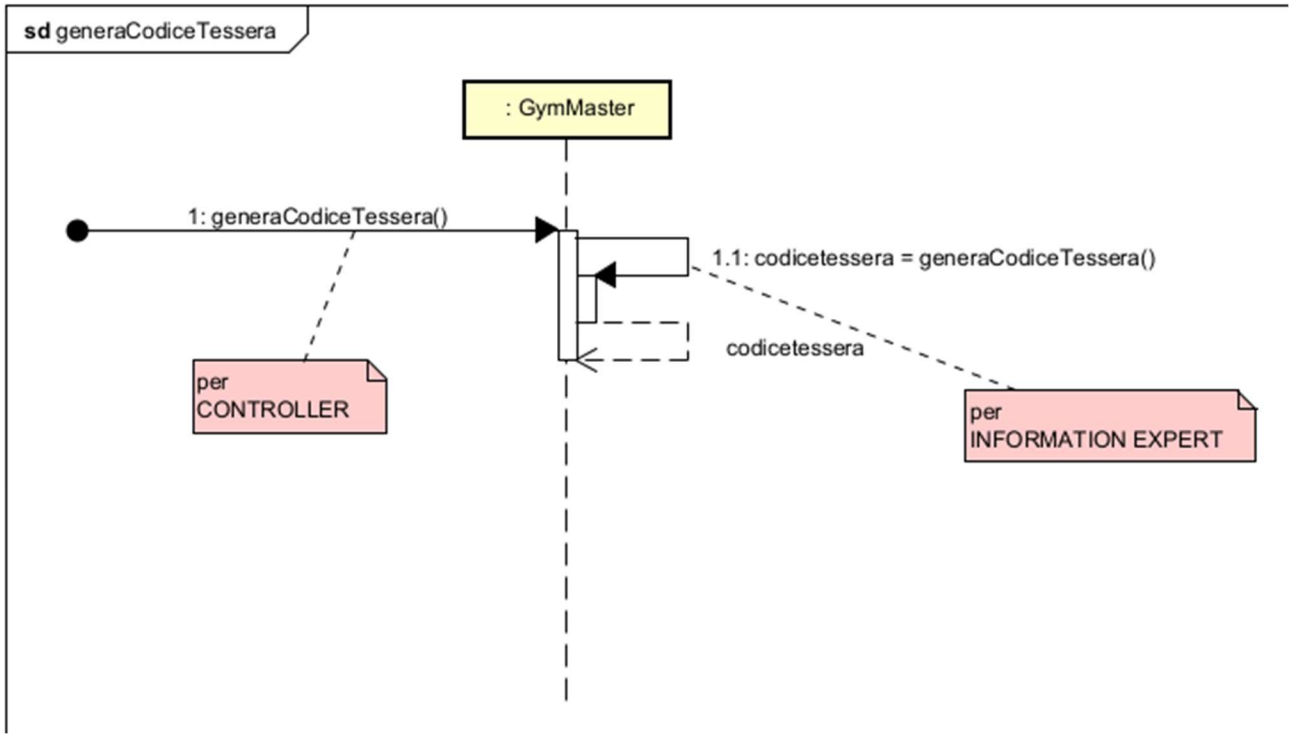
Il CalendarioLezioni implementa l'interfaccia Observer e viene registrato automaticamente su ogni nuova lezione al momento della sua creazione. Da quel momento in poi, ogni variazione dei posti disponibili viene propagata automaticamente al calendario senza alcun intervento manuale.

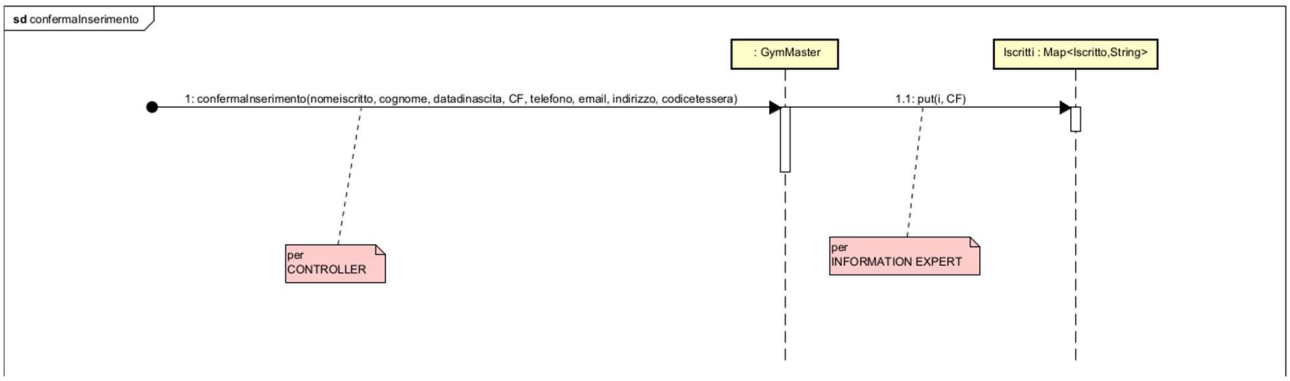
3.1 Diagramma di Sequenza

SD ITERAZIONE 1:

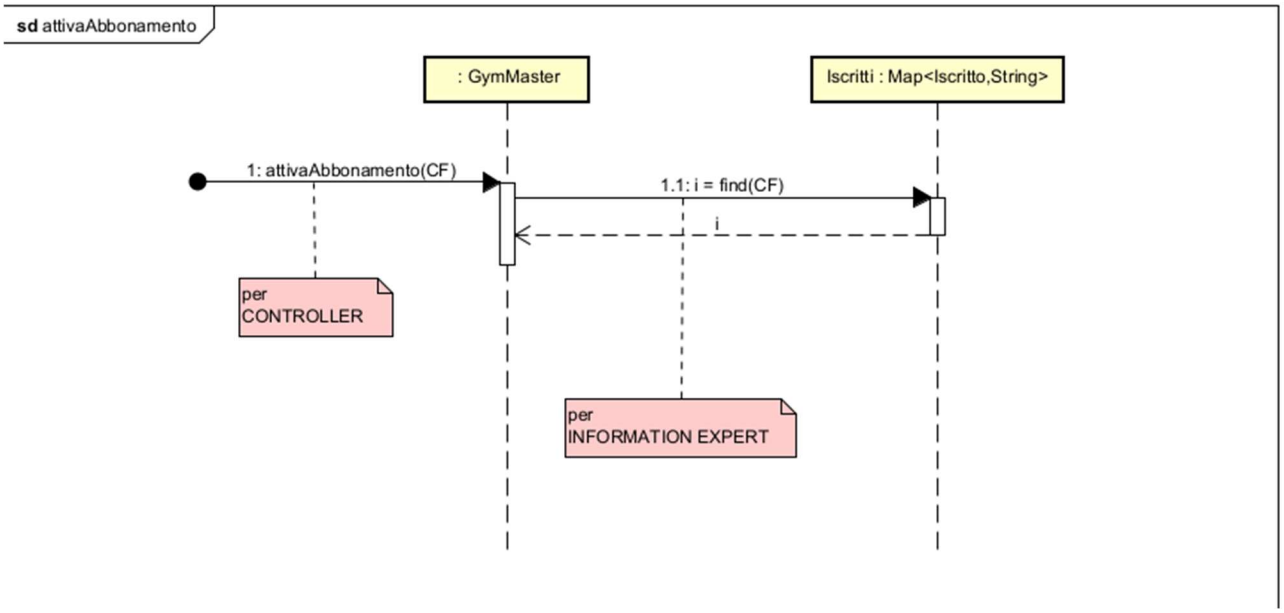
3.1.1 – SD UC1

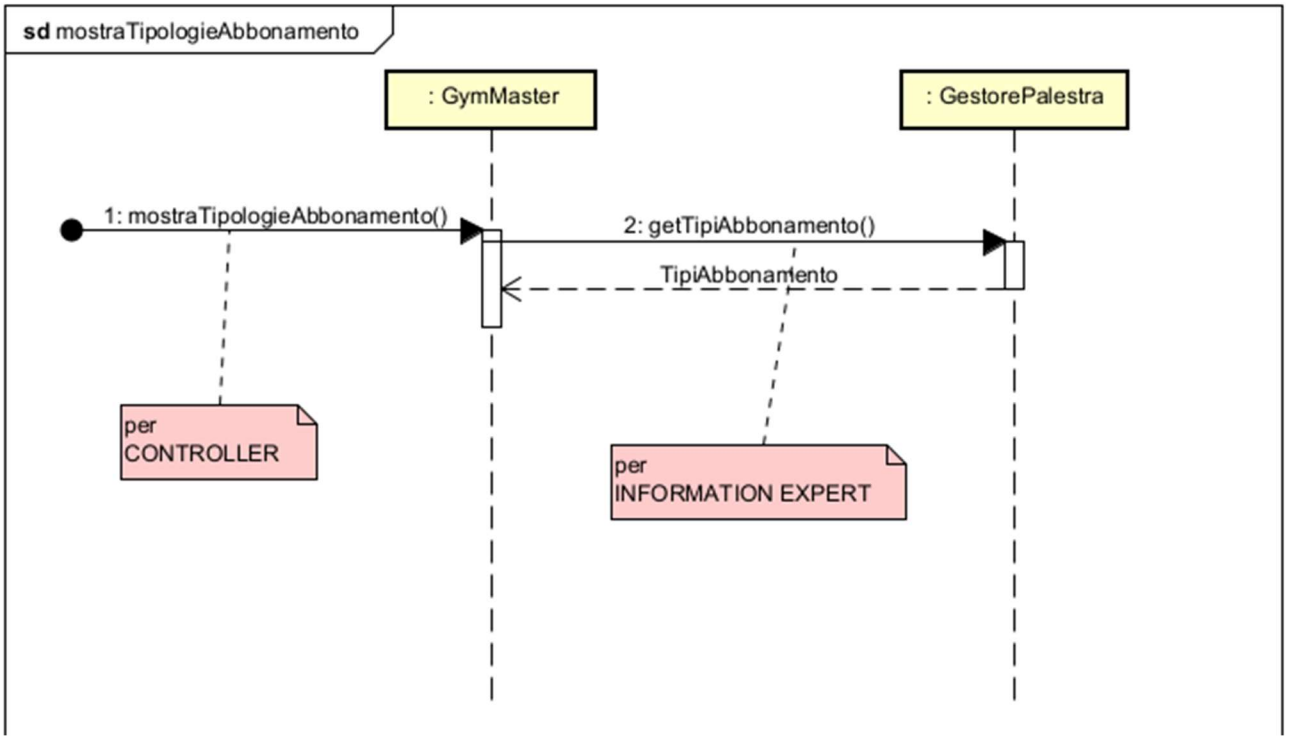


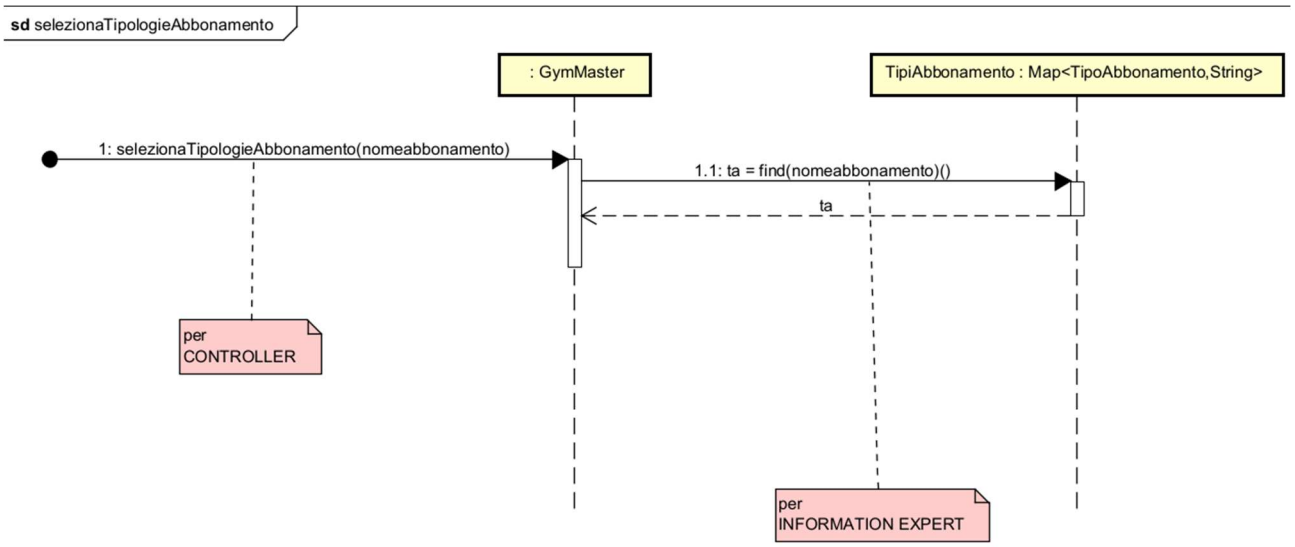

sd generaCodiceTessera
: GymMaster
1: generaCodiceTessera()
1.1: codicetessera = generaCodiceTessera()
codicetessera
per
CONTROLLER
per
INFORMATION EXPERT

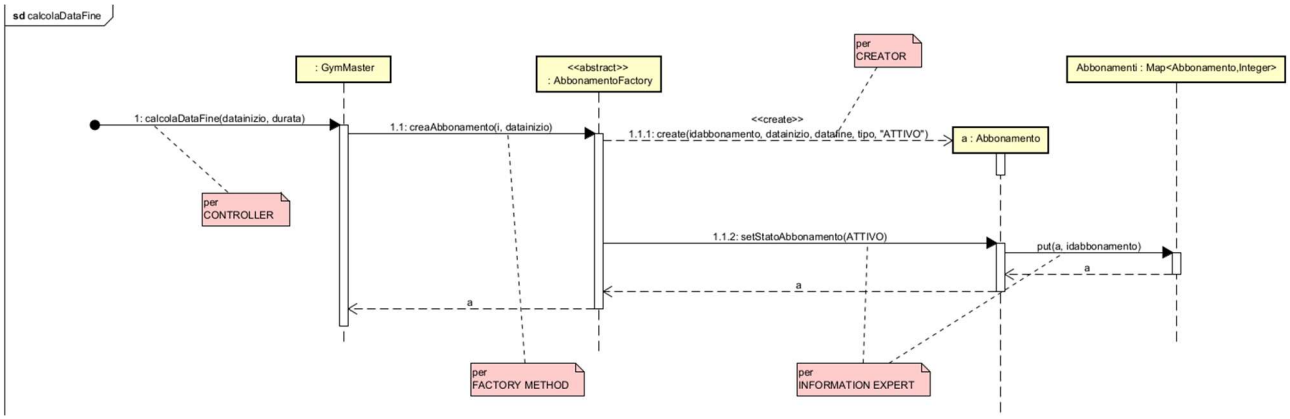

sd confermaInserimento
: GymMaster
Iscritti : Map<Iscritto,String>
1: confermaInserimento(nomeiscritto, cognome, datadinascita, CF, telefono, email, indirizzo, codicetessera)
1.1: put(i, CF)
per
CONTROLLER
per
INFORMATION EXPERT

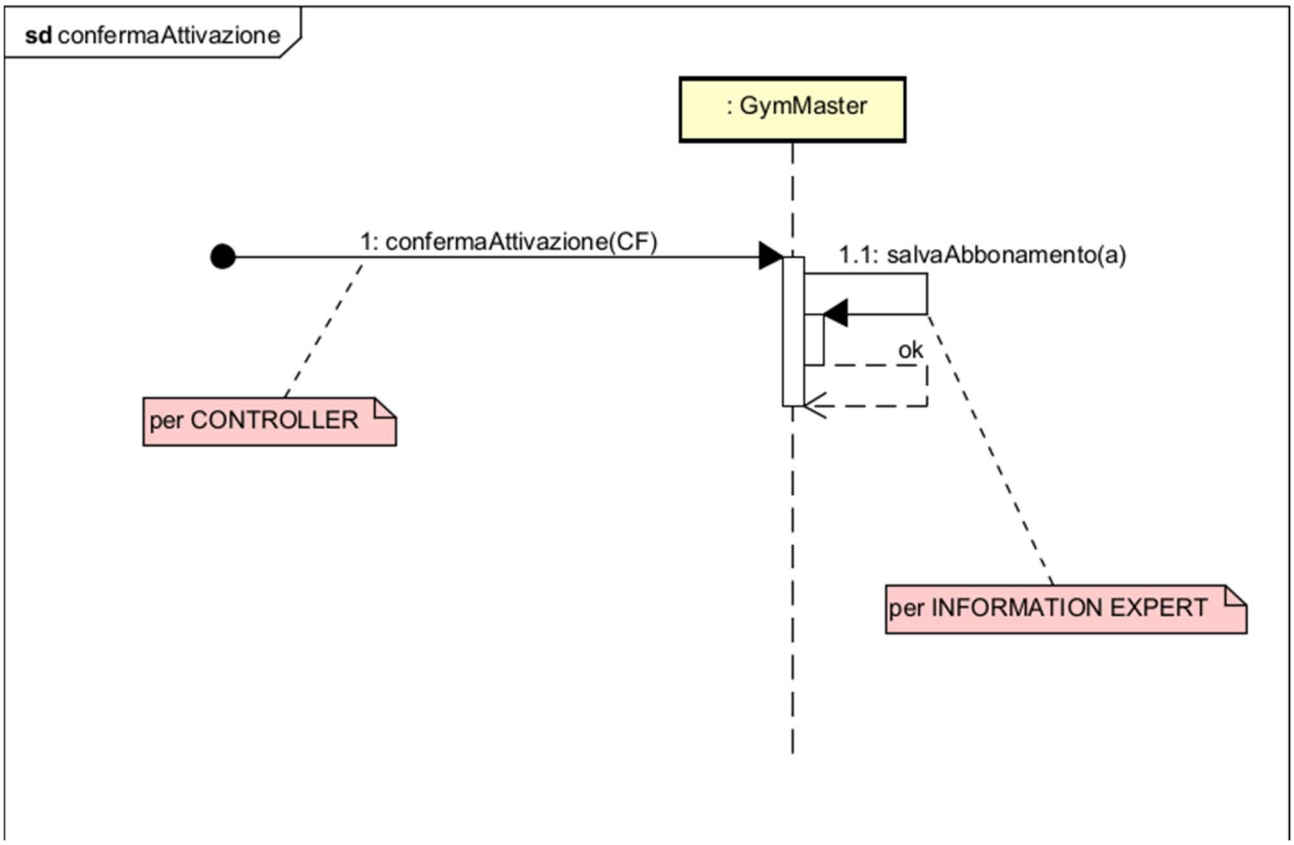
3.1.2 – SD UC2


sd attivaAbbonamento
: GymMaster
Iscritti : Map<Iscritto,String>
1: attivaAbbonamento(CF)
1.1: i = find(CF)
i
per
CONTROLLER
per
INFORMATION EXPERT

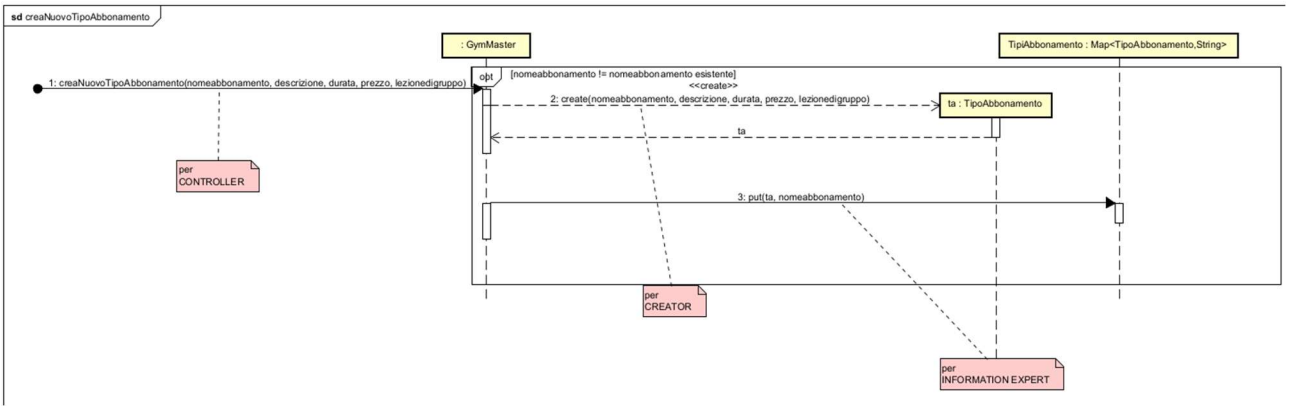

sd mostraTipologieAbbonamento
: GymMaster
: GestorePalestra
1: mostraTipologieAbbonamento()
2: getTipiAbbonamento()
TipiAbbonamento
per
CONTROLLER
per
INFORMATION EXPERT

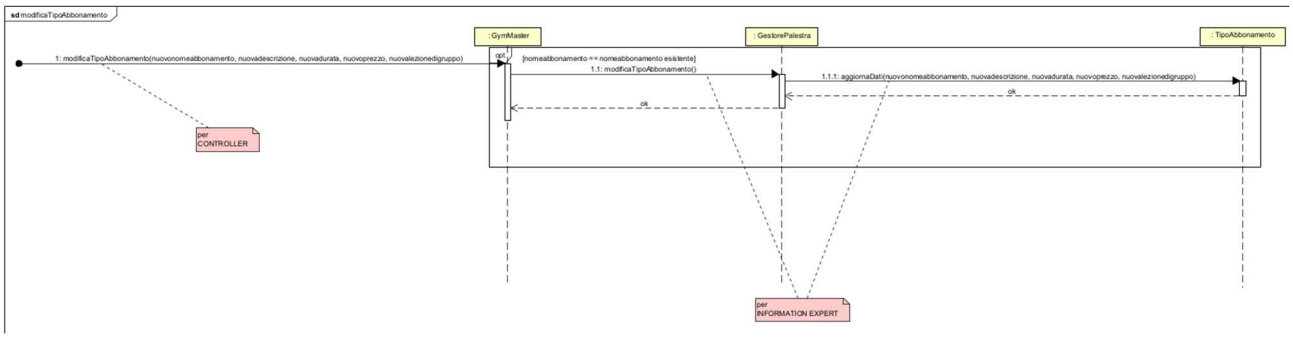

sd selezionaTipologieAbbonamento
: GymMaster
TipiAbbonamento : Map<TipoAbbonamento,String>
1: selezionaTipologieAbbonamento(nomeabbonamento)
1.1: ta = find(nomeabbonamento)()
ta
per
CONTROLLER
per
INFORMATION EXPERT


sd calcolaDataFine
: GymMaster
<<abstract>>
: AbbonamentoFactory
per
CREATOR
Abbonamenti : Map<Abbonamento,Integer>
1: calcolaDataFine(datainizio, durata)
1.1: creaAbbonamento(i, datainizio)
<<create>>
1.1.1: create(idabbonamento, datainizio, datafine, tipo, "ATTIVO")
a : Abbonamento
per
CONTROLLER
1.1.2: setStatoAbbonamento(ATTIVO)
put(a, idabbonamento)
a
a
a
per
FACTORY METHOD
per
INFORMATION EXPERT


sd confermaAttivazione
: GymMaster
1: confermaAttivazione(CF)
1.1: salvaAbbonamento(a)
ok
per CONTROLLER
per INFORMATION EXPERT

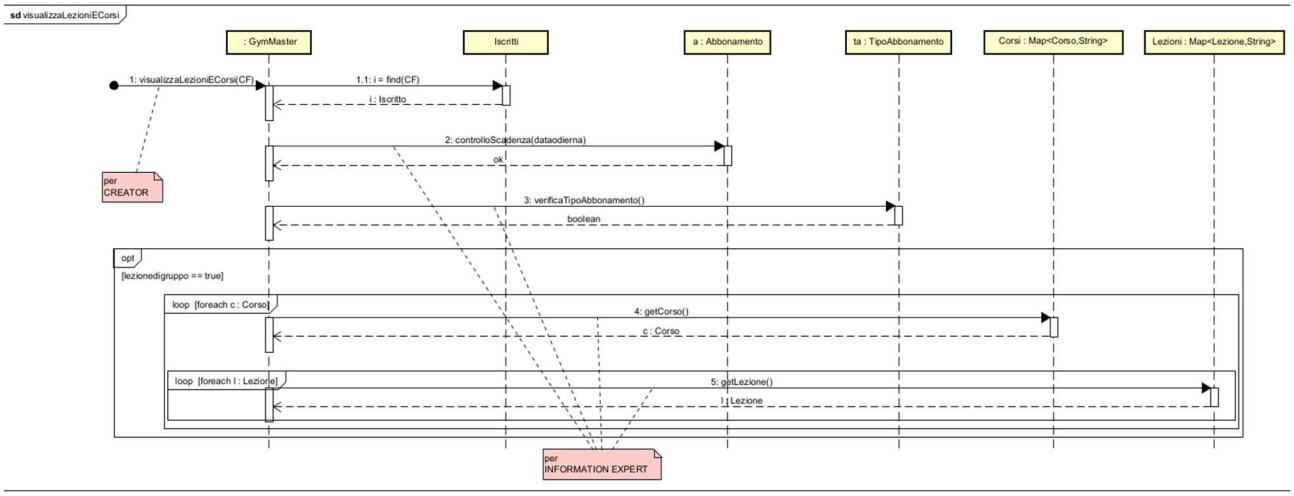
3.1.3 – SD UC9

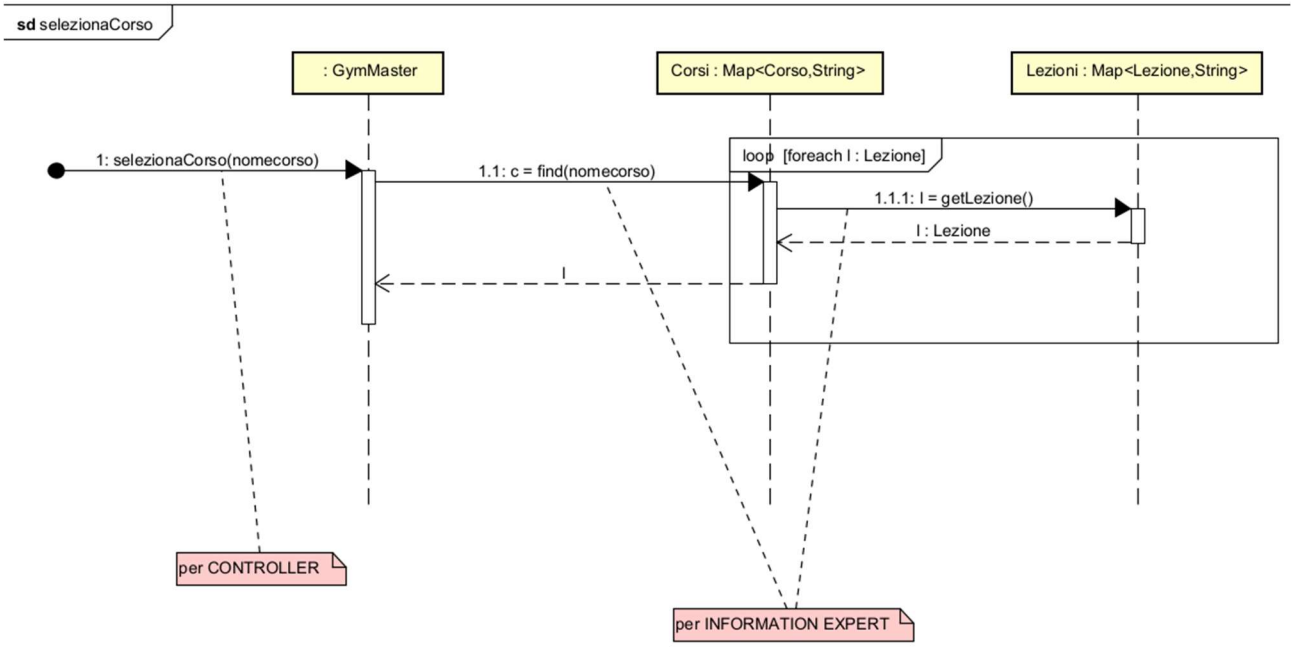

sd creaNuovoTipoAbbonamento
: GymMaster
TipiAbbonamento : Map<TipoAbbonamento,String>
1: creaNuovoTipoAbbonamento(nomeabbonamento, descrizione, durata, prezzo, lezionedigruppo)
opt
[nomeabbonamento != nomeabbonamento esistente]
<<create>>
2: create(nomeabbonamento, descrizione, durata, prezzo, lezionedigruppo)
ta : TipoAbbonamento
ta
3: put(ta, nomeabbonamento)
per CONTROLLER
per CREATOR
per INFORMATION EXPERT

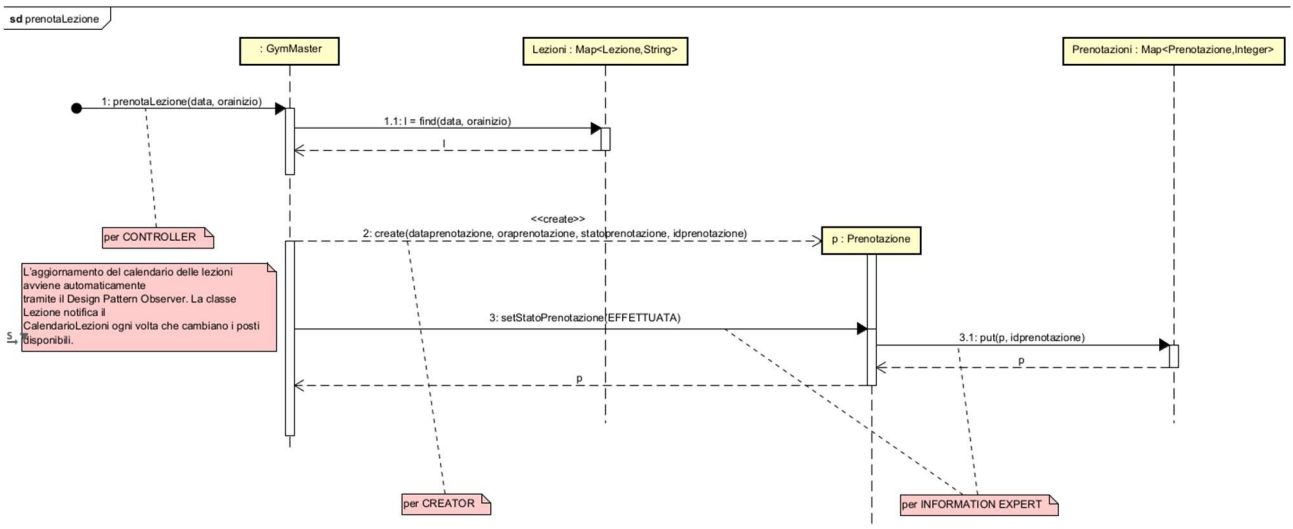

sd modificaTipoAbbonamento
: GymMaster
: GestorePalestra
: TipoAbbonamento
1: modificaTipoAbbonamento(nuovonomeabbonamento, nuovadescrizione, nuovadurata, nuovoprezzo, nuovalezionedigruppo)
opt
[nomeabbonamento == nomeabbonamento esistente]
1.1: modificaTipoAbbonamento()
1.1.1: aggiornaDati(nuovonomeabbonamento, nuovadescrizione, nuovadurata, nuovoprezzo, nuovalezionedigruppo)
ok
ok
per CONTROLLER
per INFORMATION EXPERT

SD ITERAZIONE 2:

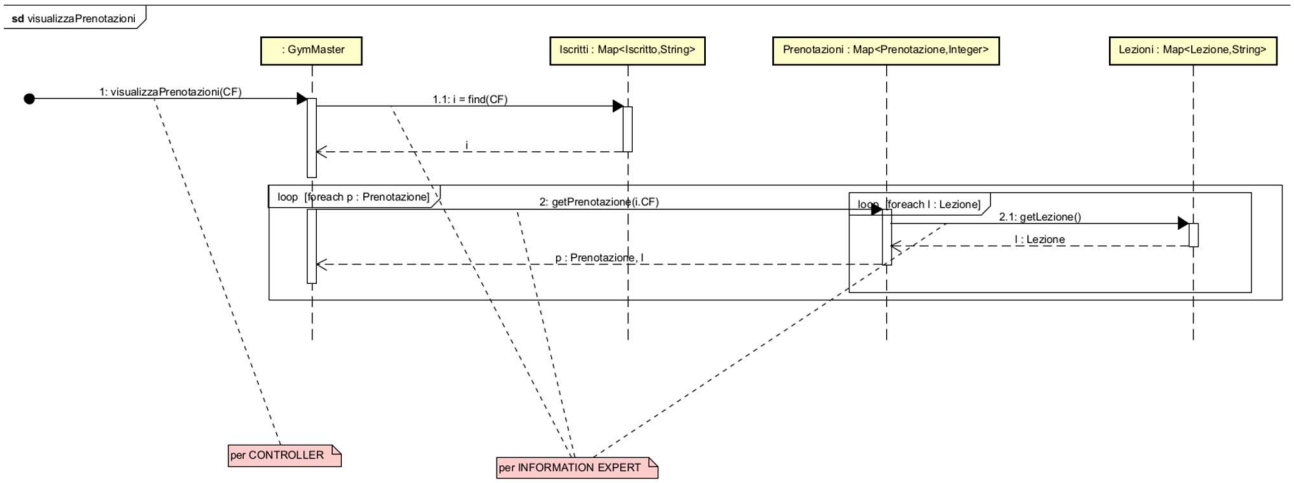
3.1.3 – SD UC3

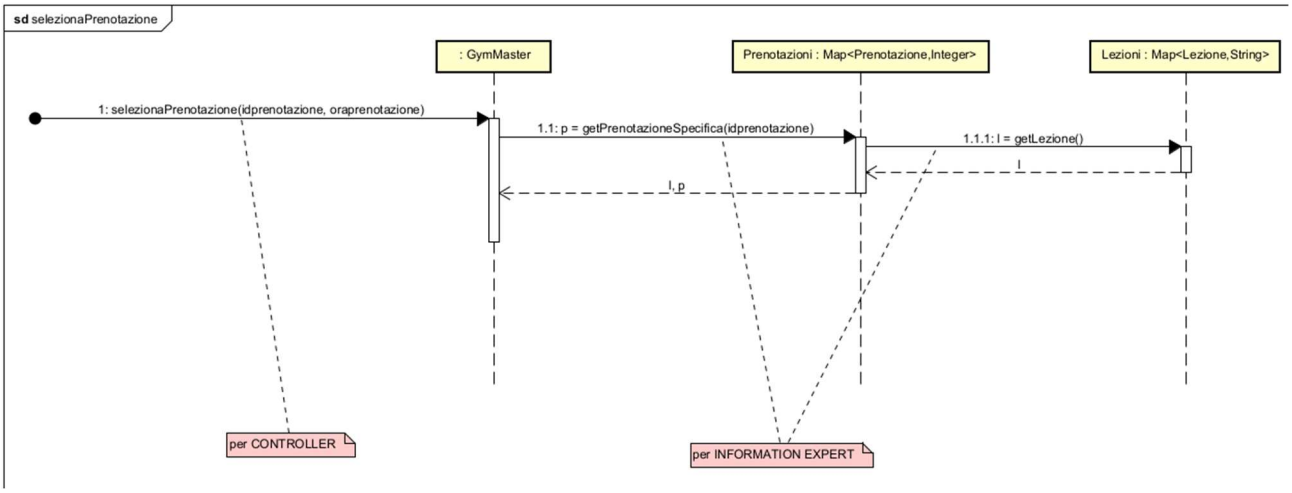


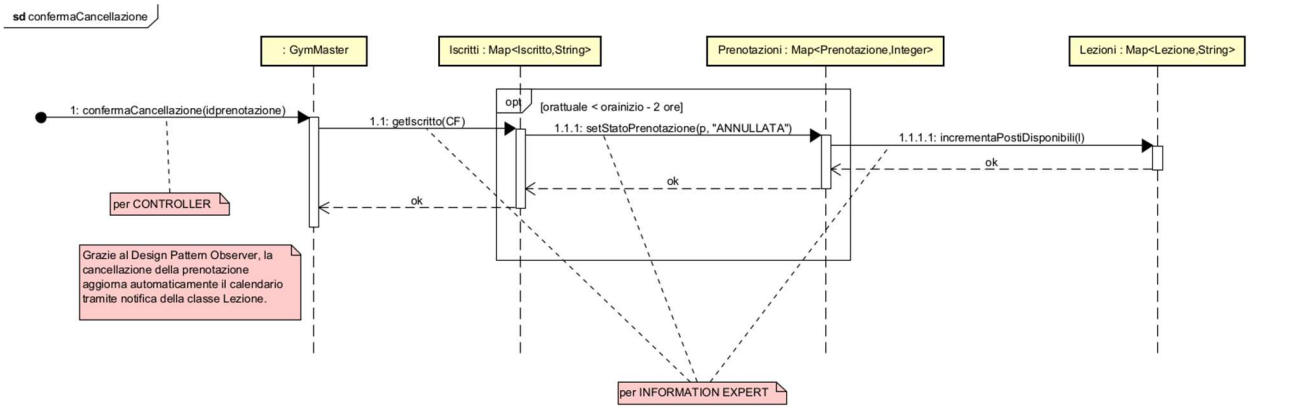




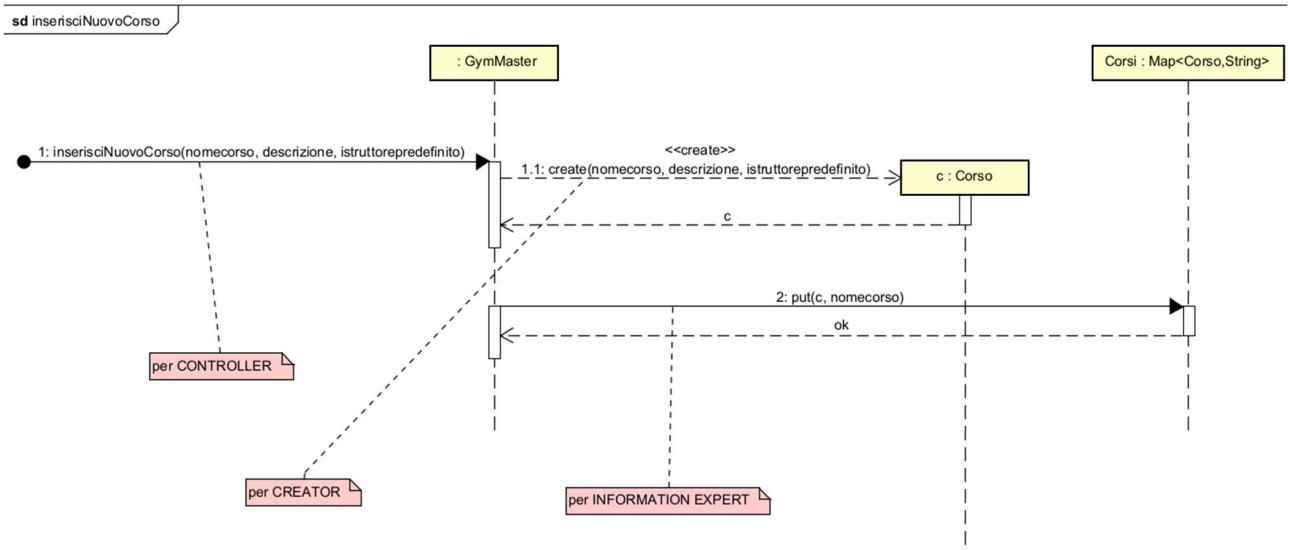
3.1.4 – SD UC4

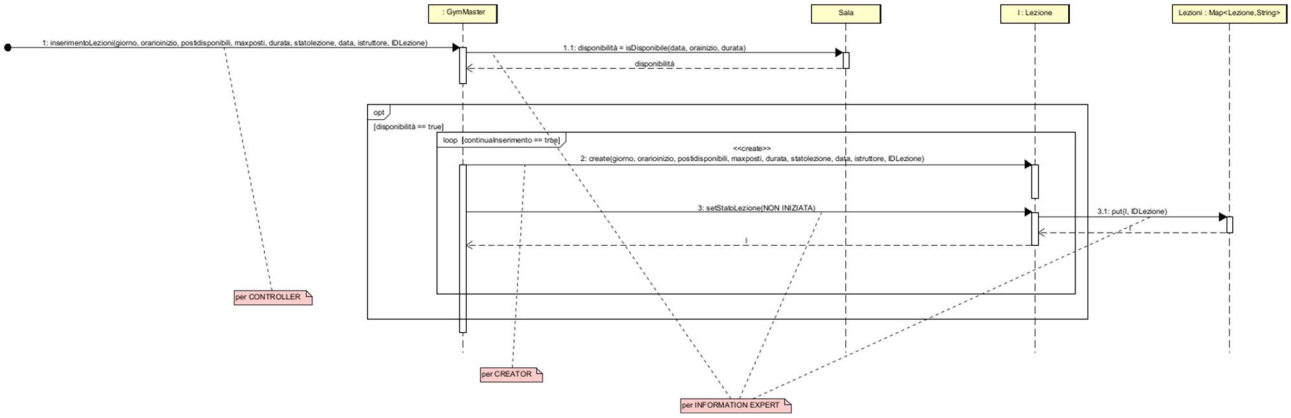

sd visualizzaPrenotazioni
: GymMaster
Iscritti : Map<Iscritto,String>
Prenotazioni : Map<Prenotazione,Integer>
Lezioni : Map<Lezione,String>
1: visualizzaPrenotazioni(CF)
1.1: i = find(CF)
i
loop [foreach p : Prenotazione]
2: getPrenotazione(i.CF)
loop [foreach l : Lezione]
2.1: getLezione()
l : Lezione
p : Prenotazione, l
per CONTROLLER
per INFORMATION EXPERT


sd selezionaPrenotazione
: GymMaster
Prenotazioni : Map<Prenotazione,Integer>
Lezioni : Map<Lezione,String>
1: selezionaPrenotazione(idprenotazione, oraprenotazione)
1.1: p = getPrenotazioneSpecifica(idprenotazione)
1.1.1: l = getLezione()
l
l, p
per CONTROLLER
per INFORMATION EXPERT

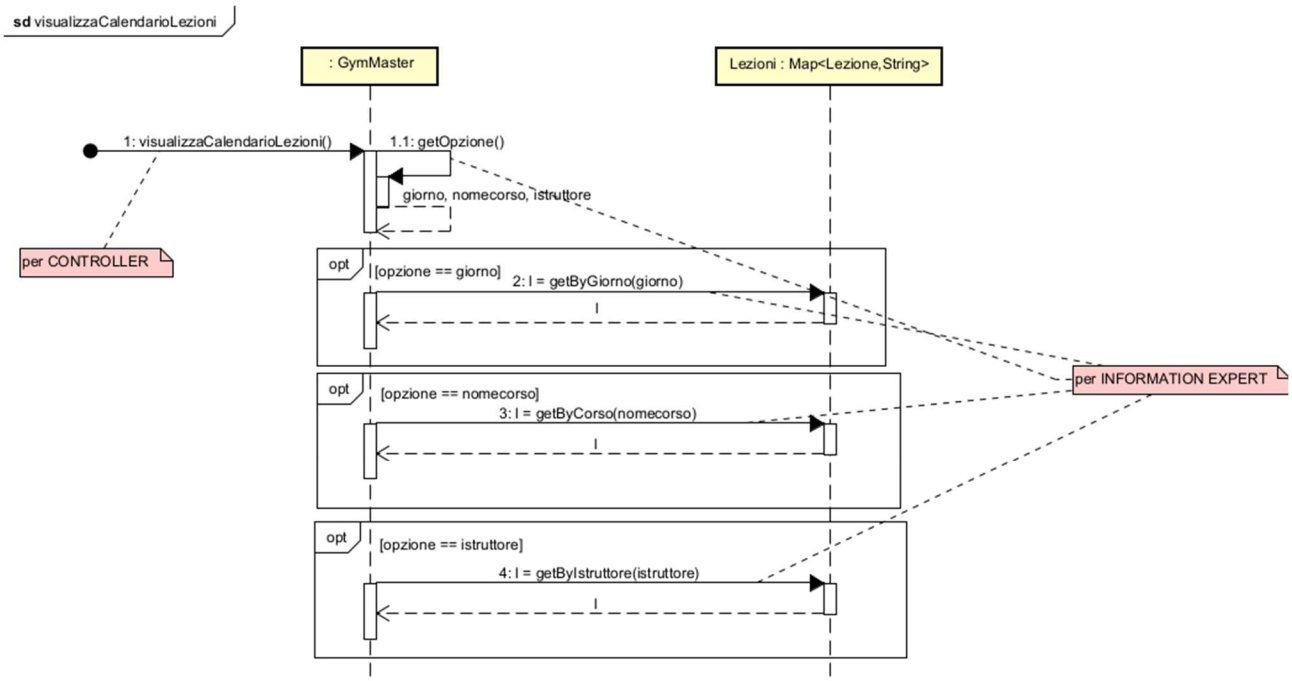

sd confermaCancellazione
: GymMaster
Iscritti : Map<Iscritto,String>
Prenotazioni : Map<Prenotazione,Integer>
Lezioni : Map<Lezione,String>
1: confermaCancellazione(idprenotazione)
opt
[orattuale < orainizio - 2 ore]
1.1: getIscritto(CF)
1.1.1: setStatoPrenotazione(p, "ANNULLATA")
1.1.1.1: incrementaPostiDisponibili(l)
ok
ok
ok
per CONTROLLER
Grazie al Design Pattern Observer, la cancellazione della prenotazione aggiorna automaticamente il calendario tramite notifica della classe Lezione.
per INFORMATION EXPERT

3.1.5 – SD UC7



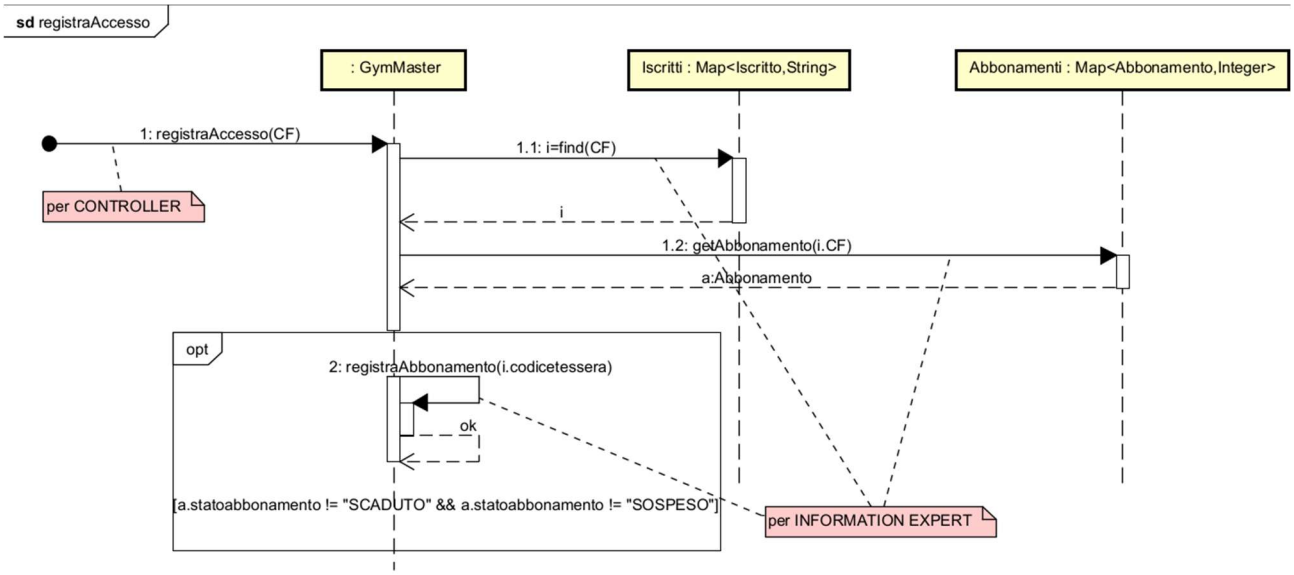


3.1.6 – SD UC8

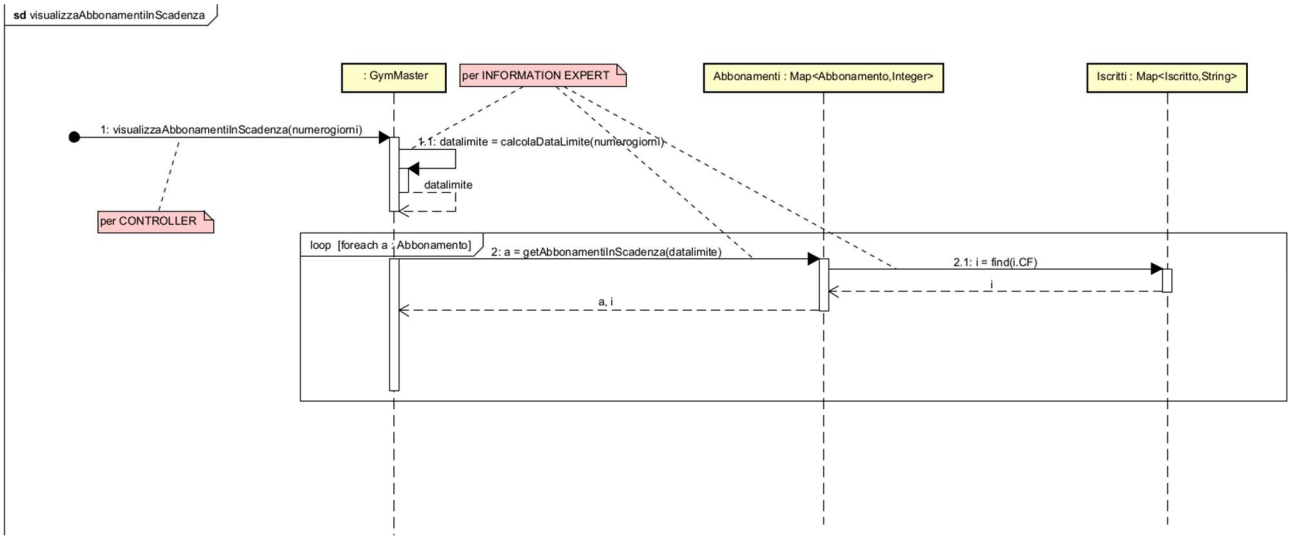


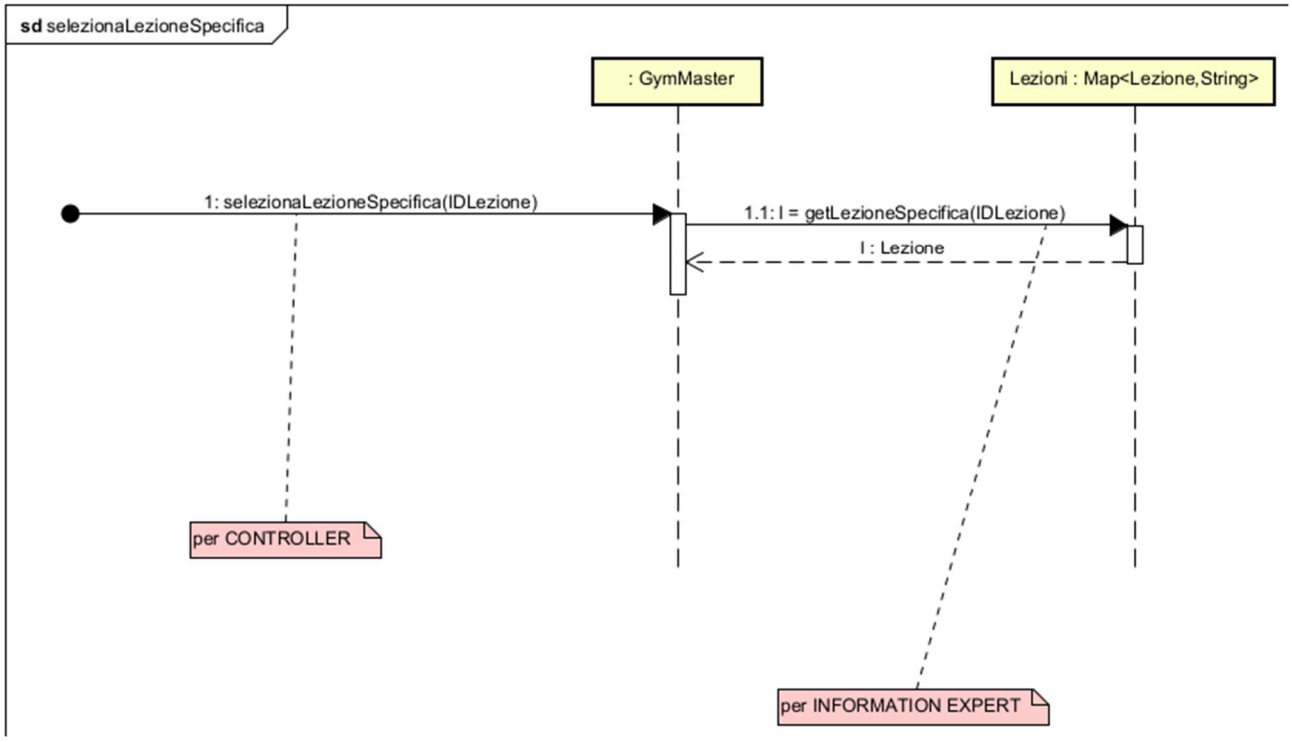
SD ITERAZIONE 3:

3.1.7 – SD UC5



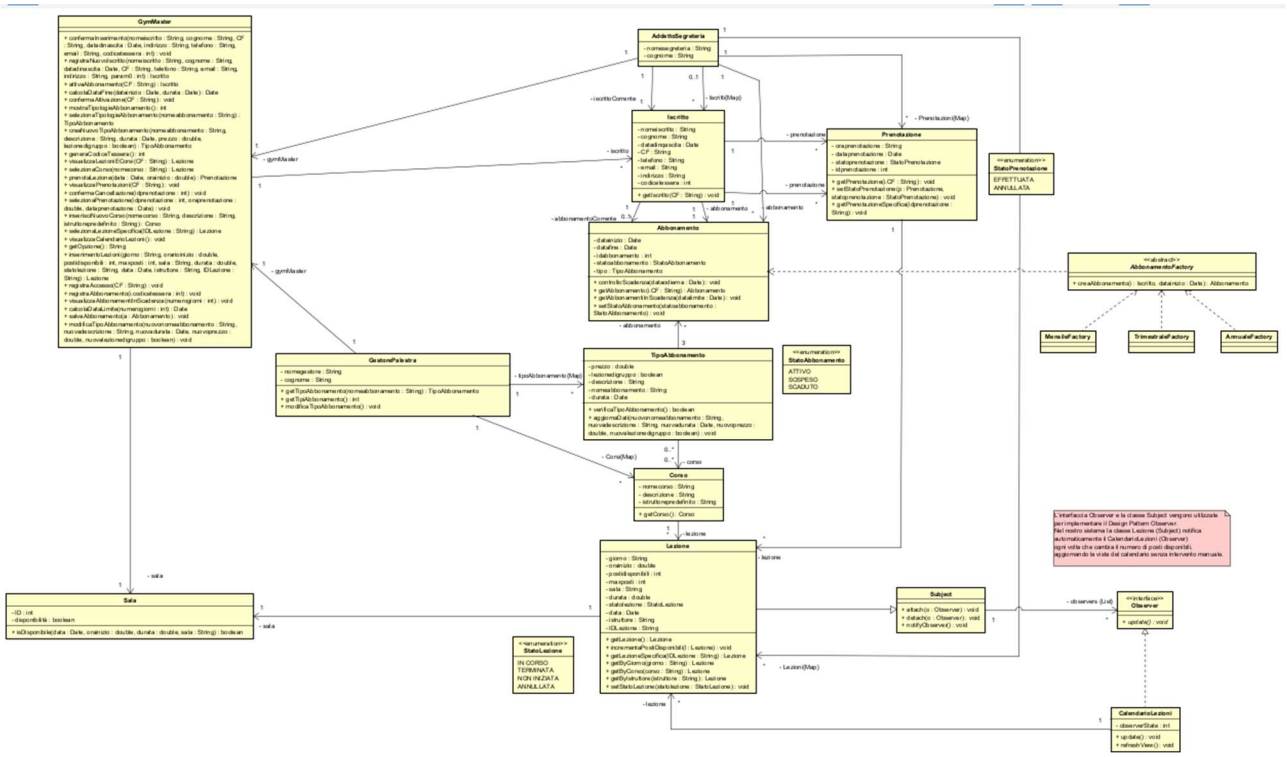
3.1.8 – SD UC6





3.2 Diagramma delle Classi

Alla fine delle tre iterazioni, il diagramma delle classi di progetto è il seguente:



4. Testing

Nel corso dello sviluppo sono stati realizzati test unitari tramite JUnit, con l'obiettivo di verificare il corretto funzionamento dei metodi considerati critici per la stabilità del sistema. I primi test hanno evidenziato l'assenza di controlli sui dati inseriti dall'utente: il sistema accettava input non validi senza generare errori. Per questo motivo, nelle iterazioni successive sono state introdotte eccezioni specifiche e meccanismi di validazione più rigorosi.

Ad ogni iterazione sono stati testati i metodi delle nuove classi introdotte e sono stati corretti eventuali malfunzionamenti relativi a iterazioni precedenti, emersi durante questa fase.

Nell'ultima iterazione, a seguito dell'introduzione dei pattern GoF **Factory Method** e **Observer**, sono stati aggiornati i test esistenti delle classi GymMasterTest e LezioneTest, e sono state create nuove classi di test dedicate alla gerarchia delle Factory (AbbonamentoFactoryTest, MensileFactoryTest, TrimestraleFactoryTest, AnnualeFactoryTest) e al pattern Observer (ObserverTest, SubjectTest, CalendarioLezioniTest).

Test implementati

Sono stati sviluppati test unitari per tutte le classi e per i metodi più significativi del sistema. Di seguito una sintesi dei principali controlli effettuati.

GymMaster

- **registraNuovoIscritto**: verifica la corretta creazione e registrazione dell'iscritto; controllo sulla duplicazione del CF.
- **generaCodiceTessera**: verifica che il codice generato rientri nel range 10000–99999.
- **attivaAbbonamento**: verifica la ricerca dell'iscritto tramite CF; gestione di CF inesistenti; impedimento dell'attivazione multipla.
- **confermaAttivazione**: verifica la corretta creazione dell'abbonamento tramite Factory Method con stato ATTIVO; verifica che la durata sia coerente con la factory selezionata (30 giorni per MensileFactory, 90 per TrimestraleFactory, 365 per AnnualeFactory); verifica il fallimento in caso di factory non istanziata, tipologia non selezionata o iscritto mancante.
- **selezionaTipologieAbbonamento**: verifica la corretta selezione di un tipo esistente e l'istanziazione automatica della factory corrispondente (MensileFactory, TrimestraleFactory o AnnualeFactory); verifica che per tipologie non riconosciute la factory resti null; restituzione di null per tipi inesistenti.
- **creaNuovoTipoAbbonamento**: verifica l'istanza e l'inserimento nella mappa dei tipi disponibili.
- **salvaAbbonamento**: verifica il corretto inserimento dell'abbonamento nella mappa.

- **getAbbonamentoFactory / setAbbonamentoFactory**: verifica il corretto accesso e impostazione della factory.
- **inserisciNuovoCorso**: verifica la creazione e l'inserimento del corso nella mappa.
- **inserimentoLezioni**: verifica la corretta creazione e associazione delle lezioni al corso; verifica che il CalendarioLezioni venga automaticamente registrato come Observer su ogni nuova lezione creata.
- **prenotaLezione**: verifica la creazione della prenotazione con stato EFFETTUATA, il decremento dei posti disponibili e la notifica automatica al CalendarioLezioni tramite il pattern Observer; verifica che prenotazioni multiple incrementino correttamente il contatore di notifiche dell'Observer.
- **confermaCancellazione**: verifica l'annullamento corretto; impedimento della doppia cancellazione.
- **visualizzaPrenotazioni**: verifica la restituzione delle prenotazioni dell'iscritto; gestione del caso senza prenotazioni.
- **registraAccesso**: verifica l'accesso consentito solo con abbonamento attivo; negazione per abbonamenti scaduti o sospesi.
- **visualizzaAbbonamentiInScadenza**: verifica l'individuazione corretta degli abbonamenti prossimi alla scadenza.
- **getCalendarioLezioni / setCalendarioLezioni**: verifica che il CalendarioLezioni venga correttamente inizializzato nel costruttore e che sia accessibile tramite getter/setter.
- **Observer su più lezioni**: verifica che tutte le lezioni create condividano lo stesso CalendarioLezioni come Observer e che le notifiche provenienti da lezioni diverse vengano correttamente accumulate.

AbbonamentoFactory (classe astratta)

- **istanziabilità**: verifica che le tre sottoclassi (MensileFactory, TrimestraleFactory, AnnualeFactory) siano correttamente istanze di AbbonamentoFactory.
- **polimorfismo**: verifica che tutte le factory, invocate tramite il riferimento astratto AbbonamentoFactory, creino abbonamenti validi con stato ATTIVO, CF iscritto correttamente associato e date di inizio e fine coerenti.
- **durate diverse**: verifica che le tre factory producano abbonamenti con durate crescenti (mensile < trimestrale < annuale).
- **associazione CF iscritto**: verifica che il codice fiscale dell'iscritto venga correttamente propagato all'abbonamento creato.

MensileFactory

- **creaAbbonamento**: verifica la corretta creazione dell'abbonamento con stato ATTIVO e CF iscritto associato.
- **durata 30 giorni**: verifica che la differenza tra data fine e data inizio sia di circa 30 giorni (con tolleranza di 1 giorno per il cambio ora legale).
- **data inizio**: verifica che la data di inizio corrisponda a quella passata come parametro.
- **data fine dopo data inizio**: verifica che la data di fine sia successiva alla data di inizio.
- **ID incrementale**: verifica che creazioni multiple producano abbonamenti con ID crescenti.
- **ereditarietà**: verifica che MensileFactory sia un'istanza di AbbonamentoFactory.

TrimestraleFactory

- **creaAbbonamento**: verifica la corretta creazione dell'abbonamento con stato ATTIVO e CF iscritto associato.
- **durata 90 giorni**: verifica che la differenza tra data fine e data inizio sia di circa 90 giorni (con tolleranza di 1 giorno per il cambio ora legale CET → CEST).
- **data inizio**: verifica che la data di inizio corrisponda a quella passata come parametro.
- **data fine dopo data inizio**: verifica che la data di fine sia successiva alla data di inizio.
- **ID incrementale**: verifica che creazioni multiple producano abbonamenti con ID crescenti.
- **ereditarietà**: verifica che TrimestraleFactory sia un'istanza di AbbonamentoFactory.
- **confronto con MensileFactory**: verifica che la durata trimestrale sia superiore a quella mensile.

AnnualeFactory

- **creaAbbonamento**: verifica la corretta creazione dell'abbonamento con stato ATTIVO e CF iscritto associato.
- **durata 365 giorni**: verifica che la differenza tra data fine e data inizio sia di circa 365 giorni (con tolleranza di 1 giorno per il cambio ora legale).
- **data inizio**: verifica che la data di inizio corrisponda a quella passata come parametro.
- **data fine dopo data inizio**: verifica che la data di fine sia successiva alla data di inizio.
- **ID incrementale**: verifica che creazioni multiple producano abbonamenti con ID crescenti.
- **ereditarietà**: verifica che AnnualeFactory sia un'istanza di AbbonamentoFactory.

- **confronto con TrimestraleFactory**: verifica che la durata annuale sia superiore a quella trimestrale.
- **non scaduto appena creato**: verifica che un abbonamento annuale appena creato risulti valido al controllo scadenza.
- **scaduto dopo data fine**: verifica che un abbonamento annuale risulti scaduto dopo 400 giorni dalla creazione.

Observer (interfaccia)

- **implementazione**: verifica che CalendarioLezioni implementi correttamente l'interfaccia Observer.
- **observer custom**: verifica che un'implementazione anonima dell'interfaccia funzioni correttamente senza generare eccezioni.

Subject

- **attach**: verifica l'aggiunta corretta di un Observer alla lista; verifica l'aggiunta di più Observer; verifica che non vengano aggiunti duplicati; verifica la gestione di null.
- **detach**: verifica la rimozione corretta di un Observer; verifica che la rimozione di un Observer non presente non generi errori; verifica la gestione di null.
- **notifyObserver**: verifica che tutti gli Observer registrati vengano notificati; verifica che notifiche multiple incrementino lo stato di ciascun Observer; verifica che la notifica senza Observer registrati non generi eccezioni.
- **getObservers**: verifica che la lista sia inizialmente vuota.

CalendarioLezioni

- **creazione**: verifica la corretta inizializzazione con observerState a zero.
- **update**: verifica che ogni chiamata a update() incrementi l'observerState di 1; verifica il corretto funzionamento con update multipli.
- **refreshView**: verifica che il metodo non generi eccezioni.
- **implementazione Observer**: verifica che CalendarioLezioni sia un'istanza di Observer.

Lezione (con test Observer)

- **costruttore**: verifica l'inizializzazione corretta degli attributi.
- **decrementaPostiDisponibili**: verifica il decremento corretto; verifica che la notifica venga inviata automaticamente al CalendarioLezioni; verifica che nessuna notifica venga inviata se i posti sono già a zero.

- **incrementaPostiDisponibili**: verifica l'incremento dopo una cancellazione; verifica che la notifica venga inviata automaticamente al CalendarioLezioni.
- **setStatoLezione**: verifica l'aggiornamento dello stato (NON_INIZIATA, IN_CORSO, TERMINATA, ANNULLATA).
- **getByGiorno**: verifica la ricerca per giorno.
- **getByIstruttore**: verifica la ricerca per istruttore.
- **ereditarietà da Subject**: verifica che Lezione sia un'istanza di Subject.
- **attach/detach su Lezione**: verifica la registrazione e rimozione del CalendarioLezioni come Observer.
- **notifiche multiple**: verifica che più decrementi producano il corretto numero di notifiche.
- **più Observer**: verifica che più CalendarioLezioni registrati sullo stesso Subject ricevano tutti la notifica.
- **Observer rimosso**: verifica che un Observer rimosso con detach() non riceva più notifiche successive.
- **lista observers vuota**: verifica che la lista degli observer sia inizialmente vuota alla creazione della Lezione.

Abbonamento

- **costruttore**: verifica l'inizializzazione corretta degli attributi.
- **creazione tramite Factory**: verifica che un abbonamento creato tramite una ConcreteFactory abbia stato ATTIVO, date coerenti e CF iscritto correttamente associato.
- **controlloScadenza**: verifica la corretta valutazione della validità; aggiornamento dello stato a SCADUTO quando necessario.
- **setStatoAbbonamento**: verifica l'aggiornamento dello stato (ATTIVO, SOSPESO, SCADUTO).
- **setTipo**: verifica la corretta impostazione del tipo abbonamento, utilizzata dalla factory dopo la creazione dell'oggetto.
- **isInScadenza**: verifica la corretta identificazione degli abbonamenti prossimi alla scadenza.
- **getAbbonamento**: verifica la ricerca tramite CF.
- **setters**: verifica l'aggiornamento di idabbonamento, datainizio, datafine.

Iscritto

- **costruttore**: verifica l'inizializzazione corretta degli attributi.
- **setCodiceTessera**: verifica l'assegnazione del codice.
- **setAbbonamentoCorrente**: verifica l'associazione dell'abbonamento.
- **getIscritto**: verifica la ricerca tramite CF.

TipoAbbonamento

- **costruttore**: verifica l'inizializzazione degli attributi.
- **verificaTipoAbbonamento**: verifica la presenza o meno delle lezioni di gruppo.
- **setters**: verifica l'aggiornamento di nome, descrizione, durata, prezzo e opzione lezioni di gruppo.

Corso

- **costruttore**: verifica l'inizializzazione degli attributi.
- **addLezione**: verifica l'aggiunta della lezione alla lista.
- **getLezioni**: verifica la restituzione della lista delle lezioni.

Prenotazione

- **setStatoPrenotazione**: verifica l'aggiornamento dello stato (EFFETTUATA, ANNULLATA).
- **setLezione**: verifica l'associazione della lezione.
- **getPrenotazione**: verifica la ricerca tramite CF.

Sala

- **isDisponibile**: verifica la disponibilità in base a data, ora e durata.

Enum (StatoAbbonamento, StatoPrenotazione, StatoLezione)

- **values**: verifica il numero corretto di valori.
- **valueOf**: verifica l'accesso ai valori tramite nome.

L'intera suite di test (108 test totali) è stata eseguita con successo, confermando l'affidabilità del processo di sviluppo adottato. I test hanno permesso di individuare e correggere tempestivamente diverse problematiche, tra cui:

- la gestione di prenotazioni già annullate,
- il controllo sugli abbonamenti già attivi,
- la validazione degli input utente,

- la corretta istanziazione delle Factory in base alla tipologia selezionata,
- la gestione dei casi di errore nel Factory Method (factory null, tipologia non riconosciuta, iscritto non selezionato),
- la tolleranza nelle verifiche di durata per il cambio di ora legale (CET → CEST), che causava uno scarto di un giorno nella conversione da millisecondi a giorni,
- la corretta registrazione automatica del CalendarioLezioni come Observer su ogni nuova lezione creata,
- la propagazione delle notifiche Observer in seguito a prenotazioni e cancellazioni,
- la verifica che Observer rimossi con detach() non ricevano più notifiche successive,
- la gestione sicura di notifiche senza Observer registrati (nessuna eccezione generata).